lean-ai

Software Factory en la Era de la IA

IA Software

Organización	lean-ai
Clasificación	Generado desde sf-kb
Fecha	13 de julio de 2026
Versión	1.0

Ia En Ingenieria

IA en Ingeniería de Software: Panorama General

La inteligencia artificial ha pasado de ser un tema académico a una herramienta cotidiana para los desarrolladores. Este artículo ofrece una vista panorámica del estado actual.

Estado Actual de la IA en Software

Timeline de hitos

```
2020 ─── GPT-3 (175B params)
  │
2021 ─── GitHub Copilot (public beta)
  │
2022 ─── ChatGPT, GitHub Copilot GA
  │
2023 ─── GPT-4, Claude 2, Gemini, Llama 2
  │       Agentic coding emerge
  │
2024 ─── Claude 3, GPT-4o, o1/o3
  │       MCP protocol, multi-agent systems
  │
2025 ─── Claude Opus 4, Gemini 2.5
  │       Autonomous coding agents
  │
2026 ─── Multi-modal + reasoning agents
         Full SDLC AI integration
```

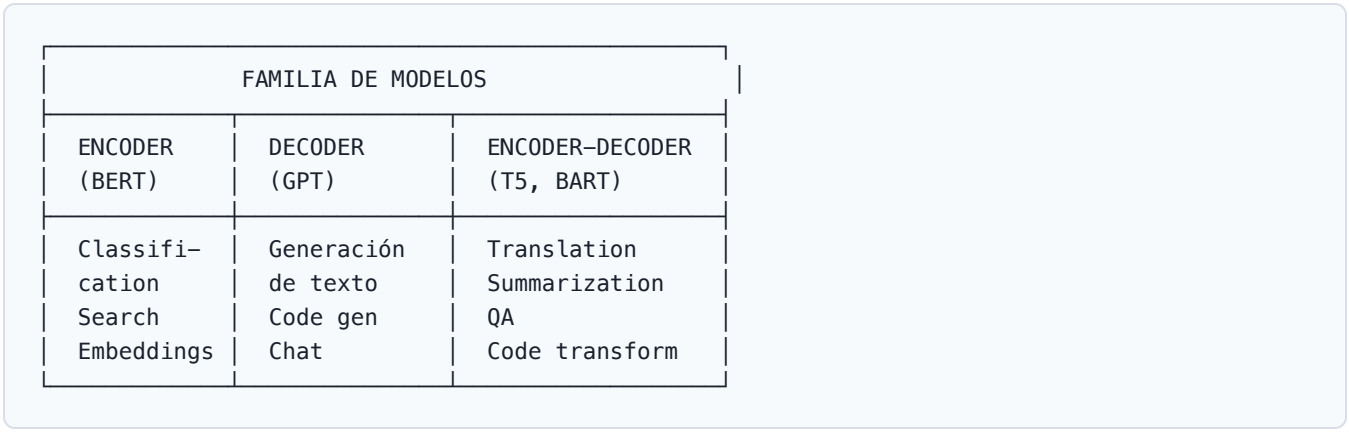
¿Qué pueden hacer los LLMs hoy?

Capacidad	Estado	Ejemplo
Generar código	Maduro	Copilot, Cursor
Explicar código	Maduro	ChatGPT, Claude
Escribir tests	Útil pero requiere review	Copilot, Codium
Code review	Emergente	CodeRabbit, PR-Agent
Diseñar arquitectura	Asistido	LLMs + contexto de repo
Debugging	Parcial	Stack traces → explicaciones
Documentación	Maduro	Auto-generate docs

Capacidad	Estado	Ejemplo
DevOps	Emergente	AIOps, auto-remediation

Modelos Fundamentales

Tipos de modelos relevantes



Modelos usados en AI coding

Modelo	Provider	Context Window	Uso principal
GPT-4o	OpenAI	128K	Copilot, general
Claude 3.5/Opus	Anthropic	200K	Cursor, long codebase
Gemini 2.5	Google	1M	Code review, large files
Llama 3	Meta	128K	Open source, self-hosted
Codestral	Mistral	32K	Code-specialized
DeepSeek Coder	DeepSeek	128K	Code-specialized, open

Áreas de Impacto

1. Desarrollo (Coding)

- **Autocomplete**: Sugerencias en tiempo real
- **Chat**: Preguntas sobre código, debugging
- **Multi-file editing**: Cambios agentic en múltiples archivos
- **Boilerplate generation**: Config, tests, scaffolding

Referencia: Generación de Código

2. Testing

- **Test generation**: Crear unit/integration tests automáticamente
- **Mutation testing**: Evaluar quality de tests
- **Test case suggestion**: Boundary cases, edge cases

Referencia: Generación de Tests

3. Code Review & Quality

- **Automated PR review**: Análisis de cambios
- **Vulnerability detection**: SAST mejorado con AI
- **Refactoring suggestions**: Code smell detection

Referencia: Análisis de Código

4. Documentación

- **Auto-doc**: Generar docs de código existente
- **API documentation**: OpenAPI specs desde código
- **README generation**: Documentación de proyectos

Referencia: IA en Documentación

5. Diseño y Arquitectura

- **Pattern detection**: Identificar patrones en código existente
- **Design suggestions**: Sugerir arquitecturas
- **ADR generation**: Architectural Decision Records

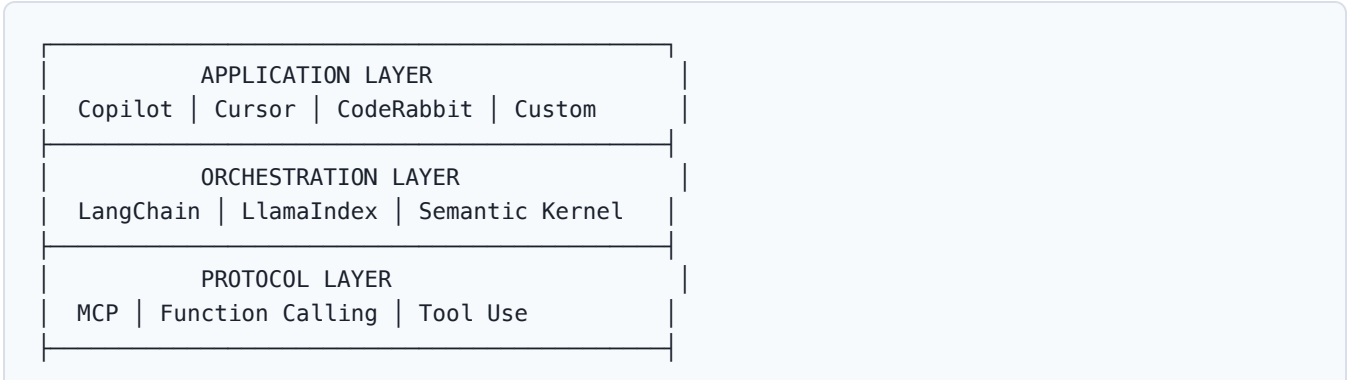
Referencia: IA en Diseño

6. DevOps y Operaciones

- **Log analysis**: Interpretar logs de error
- **Incident response**: Sugerir remediaciones
- **Infrastructure as Code**: Generar Terraform, K8s configs

Referencia: IA en DevOps

El Stack de IA para Software



```
│           MODEL LAYER                          │
│  GPT-4o │ Claude │ Gemini │ Llama │ Mistral │
├──────────────────────────────────────────────┤
│           INFRASTRUCTURE LAYER                 │
│  GPU │ API Gateway │ Vector DB │ Cache       │
└──────────────────────────────────────────────┘
```

Adopción en la Industria

Datos de adopción (2025-2026)

Métrica	Valor
Developers usando AI tools	~75%
Empresas con AI coding policy	~45%
Productividad reportada (mejora)	20-55%
Code aceptado de AI (promedio)	30-40%
Enterprises con AI budget dedicado	~60%

Perfiles de usuario

Perfil	Uso principal	Herramienta típica
Junior	Learning, boilerplate	Copilot Chat
Mid-level	Productivity boost	Copilot + Cursor
Senior	Architecture, review	Cursor + custom agents
Architect	Design, patterns	LLMs + context
DevOps	IaC, troubleshooting	Q Developer, custom
Security	Vulnerability scanning	CodeRabbit, Snyk AI

Buenas Prácticas de Adopción

Framework de adopción gradual

```
Fase 1: Experimentación (1–2 meses)
├── Pilot con 5–10 developers
├── Copilot individual licenses
└── Medir baseline + impacto

Fase 2: Estándarización (2–3 meses)
├── Team-wide rollout
```

```
├── Custom instructions
├── Acceptable use policy
└── Training sessions

Fase 3: Integración (3–6 meses)
├── AI en CI/CD pipelines
├── Code review automation
├── Custom agents
└── Metrics dashboard

Fase 4: Optimización (continuo)
├── Fine-tuning models
├── Custom tools
├── Agent orchestration
└── ROI measurement
```

Relación con otros conceptos

- Se fundamenta en la Era de la IA
- Se aplica dentro del modelo de Software Factory
- Se mide con métricas de productividad
- Se complementa con herramientas existentes
- Conlleva riesgos documentados en limitaciones y riesgos
- Impacta calidad y seguridad
- Se integra con prácticas DevOps

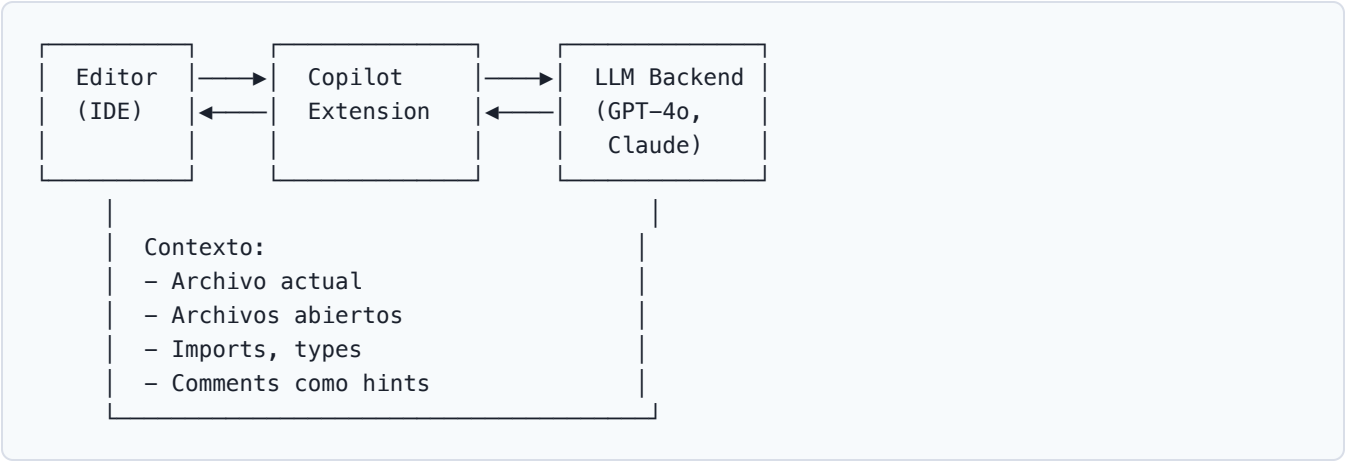
Copilot Efecto

GitHub Copilot y el Efecto Copilot

GitHub Copilot popularizó la asistencia de código con IA. Este artículo analiza cómo funciona, los datos de productividad y las limitaciones conocidas.

¿Cómo funciona Copilot?

Arquitectura simplificada



Flujo de una sugerencia

1. **Context collection**: IDE envía el código circundante
2. **Prompt construction**: Se arma el prompt con contexto
3. **Model inference**: LLM genera múltiples candidatos
4. **Ranking**: Se selecciona la mejor sugerencia
5. **Display**: Se muestra como ghost text
6. **Acceptance**: Developer acepta, edita o rechaza

Contexto que usa Copilot

Fuente de contexto	Peso en sugerencia
Código en el cursor	Alto
Archivos abiertos	Medio
Función/docstring actual	Alto
Imports y types	Medio
Comentarios	Alto

Fuente de contexto	Peso en sugerencia
Nombre del archivo	Bajo-Medio
Historial reciente	Bajo

Datos de Productividad

Estudios y métricas

Estudio/Métrica	Resultado
GitHub Study (2022)	55% faster task completion
GitHub Study (2023)	46% más código completado
Microsoft Research (2023)	30-40% acceptance rate
Stack Overflow Survey (2024)	76% quieren seguir usándolo
Metanalysis (2025)	25-55% productivity gain

Desglose por tipo de tarea

Tipo de tarea	Impacto	Notas
Boilerplate code	Alto	Config, CRUD, setup
Unit tests	Alto	Tests mecánicos
Documentation	Medio-Alto	Comments, READMEs
API integration	Medio	Calls, schemas
Bug fixing	Medio	Depende del contexto
Architecture	Bajo	Requiere contexto amplio
Complex algorithms	Bajo	AI puede generar incorrectos
Domain logic	Bajo	Requiere conocimiento del negocio

El "Efecto Copilot"

El efecto Copilot se refiere al fenómeno donde:

1. **Mayor velocidad percibida**: Developers sienten que avanzan más rápido
2. **Flow state mejorado**: Menos interrupciones para buscar en docs
3. **Onboarding acelerado**: Junior developers contribuyen antes
4. **Efecto dependencia**: Riesgo de over-reliance
5. **Calidad variable**: No toda sugerencia es correcta

```
Antes de Copilot:
  Escribir → Buscar docs → Copiar → Adaptar → Test
  [████████████████████████████] 100% tiempo

Con Copilot:
  Escribir → Aceptar sugerencia → Test → Ajustar
  [██████████████████] 70% tiempo (30% faster)
```

Planes y Pricing

Plan	Precio	Features
Free	$0	2000 completions/mes, chat
Pro	$10/mes	Unlimited completions, chat
Pro+	$39/mes	Premium models, Copilot Workspace
Business	$19/user/mes	Org management, policy controls
Enterprise	$39/user/mes	Repo indexing, audit logs, SSO

Limitaciones conocidas

1. Code Quality Issues

```
# Copilot puede sugerir código que compila pero es incorrecto

# Prompt:
def calculate_average(numbers):
    # Copilot sugiere:
    return sum(numbers) / len(numbers)  # ZeroDivisionError!

# Mejor:
def calculate_average(numbers):
    if not numbers:
        return 0
    return sum(numbers) / len(numbers)
```

2. Security Concerns

Riesgo	Ejemplo	Mitigación
Hardcoded secrets	API keys en código	Pre-commit scanning
Vulnerable patterns	SQL injection, XSS	Security linters
Outdated libraries	Deprecated functions	Dependency scanning

Riesgo	Ejemplo	Mitigación
License violations	GPL en código propietario	License checkers

3. Context Window Limitations

- No ve el proyecto completo (solo archivos abiertos)
- No entiende la arquitectura global
- No conoce business logic del dominio
- Puede contradecir decisiones previas del equipo

4. Bias y Consistency

- Sugerencias sesgadas hacia lenguajes populares
- Patrones de código no siempre idiomáticos
- Puede generar "anti-patterns" comunes
- Inconsistente en design patterns

Best Practices de Uso

Para equipos

1. **Establecer guías de uso**: Qué aceptar, qué revisar
2. **Custom instructions**: Configurar contexto del proyecto
3. **Training**: Enseñar a prompting effectively
4. **Code review**: Siempre revisar código generado
5. **Metrics**: Medir impacto real, no percibido

Para individuos

1. **Escribe buenos nombres**: Variables descriptivas → mejores sugerencias
2. **Usa docstrings**: Documenta la intención antes del código
3. **Comenta la lógica**: Hints para el modelo
4. **Rechaza frecuentemente**: No todo suggestion es buena
5. **Aprende del output**: Analiza qué sugiere para aprender

Custom Instructions (ejemplo)

```
# .github/copilot-instructions.md

## Project conventions
- Use TypeScript strict mode
- Prefer functional components (React)
- Use Zod for validation
- Follow Clean Architecture patterns
- Tests with Vitest, not Jest
```

```
## Code style
- Prefer early returns
- Use descriptive variable names
- Avoid any type
- Use named exports
```

Copilot vs Competencia

Aspecto	Copilot	Cursor	Cody
Autocomplete quality	Excelente	Excelente	Buena
Codebase context	Enterprise only	Completo	Completo
Multi-file editing	Workspace	Composer	Agentic
Pricing	$10-39/mes	$20-40/mes	$9/mes
IDE support	Multi-IDE	Solo Cursor	Multi-IDE
Enterprise features	Fuertes	Débiles	Fuertes

Referencia: AI Coding Assistants — Comparison completa

Relación con otros conceptos

- Datos de impacto: Impacto en Métricas
- Limitaciones: Limitaciones y Riesgos
- Generación de código: Generación de Código
- Pair programming: AI Pair Programming
- Code review: AI Code Review
- Productividad: Métricas de productividad

Generacion Codigo

Generación de Código con IA

La generación de código con IA va más allá del autocomplete. Incluye técnicas, métricas de calidad y la necesidad imperativa de code review.

Técnicas de Generación

Autocomplete (inline completion)

- **Trigger**: Continuación natural del código
- **Latencia**: < 300ms para ser útil
- **Contexto**: Archivo actual + archivos abiertos
- **Uso**: 60-70% de interacciones con AI

Chat-based generation

```
# Prompt: "Create a REST API endpoint for user registration
#          with email validation and password hashing"

# Output: Implementation completa con:
# - Route definition
# - Input validation (Zod/Joi)
# - Password hashing (bcrypt)
# - Error handling
# - Unit tests
```

- **Trigger**: Conversación con el LLM
- **Contexto**: Prompt + contexto de proyecto
- **Uso**: Funciones nuevas, refactoring, explicaciones

Agentic multi-file generation

```
┌──────────────────────────────────────────┐
│  "Add dark mode support to the application" │
│                                          │
│  Agent plan:                             │
│  1. Create ThemeContext.tsx              │
│  2. Create ThemeProvider.tsx             │
│  3. Update App.tsx                       │
│  4. Add theme toggle component           │
│  5. Update CSS variables                 │
│  6. Add unit tests                       │
│  7. Update documentation                 │
└──────────────────────────────────────────┘
```

- **Trigger**: Tarea de múltiples archivos
- **Contexto**: Repo completo indexado
- **Uso**: Features completas, migrations

Code transformation

```
// Input: JavaScript callback-based code
fs.readFile('file.txt', function(err, data) {
  if (err) throw err;
  console.log(data);
});

// Prompt: "Convert to async/await with error handling"

// Output:
async function readFileAsync(path) {
  try {
    const data = await fs.promises.readFile(path, 'utf-8');
    console.log(data);
    return data;
  } catch (error) {
    console.error(`Error reading ${path}:`, error);
    throw error;
  }
}
```

Calidad del Código Generado

Métricas de calidad

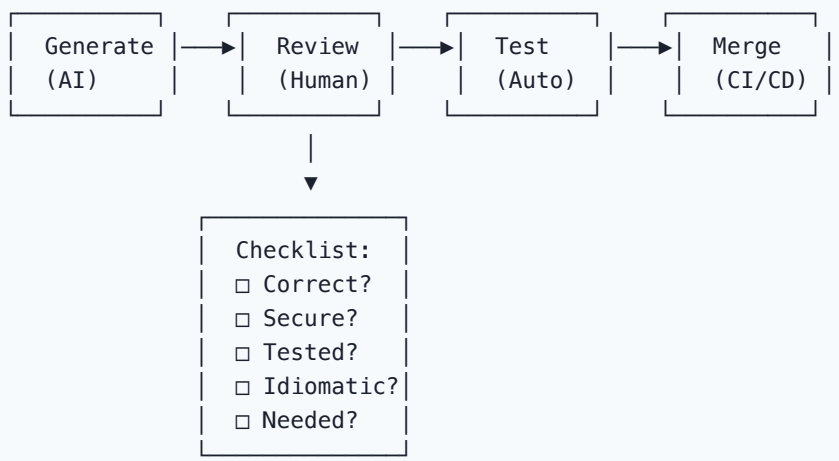
Métrica	AI-generated	Human-written	Nota
Correctness	70-85%	90-95%	Requiere review

Métrica	AI-generated	Human-written	Nota
Readability	Alta	Variable	AI tiende a ser claro
Performance	Media	Variable	No optimiza edge cases
Security	Variable	Variable	Riesgo de patterns inseguros
Maintainability	Media	Variable	Puede ser over-engineered
Test coverage	Baja	Alta	AI no always testa

Code smells en código generado

Problem	Ejemplo	Mitigación
Over-engineering	Patrones innecesarios	Simplify prompt
Missing edge cases	No maneja null/empty	Explicit requirements
Outdated patterns	Callbacks en vez de async	Specify modern practices
Generic naming	`data`, `result`, `item`	Use descriptive comments
Missing error handling	No try/catch	Request error handling
Security holes	SQL injection, XSS	Security-focused prompts

El Flujo de Code Review para AI



Code review checklist para código AI

1. **¿Es correcto?** — ¿Resuelve el problema planteado?
2. **¿Es seguro?** — ¿Maneja inputs maliciosos?
3. **¿Tiene tests?** — ¿Está cubierto por tests?

4. **¿Es idiomático?** — ¿Sigue convenciones del proyecto?
5. **¿Es necesario?** — ¿Agrega valor o es over-engineering?
6. **¿Es mantenible?** — ¿Fácil de entender y modificar?
7. **¿Es performant?** — ¿No introduce N+1 queries, etc.?

Comparison: AI vs Human Code Generation

Aspecto	AI	Humano
Velocidad	Muy rápida	Variable
Consistencia	Alta (mismo prompt = mismo output)	Variable
Creatividad	Baja (repite patrones)	Alta
Contexto de negocio	Sin contexto	Entiende el domain
Edge cases	Frecuentemente omite	Depende de experiencia
Innovación	Copia soluciones existentes	Puede innovar
Costo por línea	Bajo	Alto
Escalabilidad	Lineal con prompts	Limitada por tiempo

Herramientas de Generación

Herramienta	Tipo	Contexto	Multi-file
GitHub Copilot	IDE extension	Archivos abiertos	Workspace
Cursor Composer	AI-native IDE	Repo completo	
Cody + Commands	IDE extension	Code graph	
ChatGPT/Claude	Web/API	Prompt only	❌
Aider	CLI tool	Repo (git)	
Continue.dev	IDE extension	Configurable	Parcial

Referencia: AI Coding Assistants

Prompt Templates para Code Generation

Template: Nueva función

```
Create a [language] function called [name] that:
- Takes [parameters] as input
- Returns [output type]
- Should handle [edge cases]
```

```
- Use [library/framework] for [specific concern]
- Follow [pattern] pattern
- Include error handling for [scenarios]
- Add JSDoc/docstring comments
```

Template: Refactoring

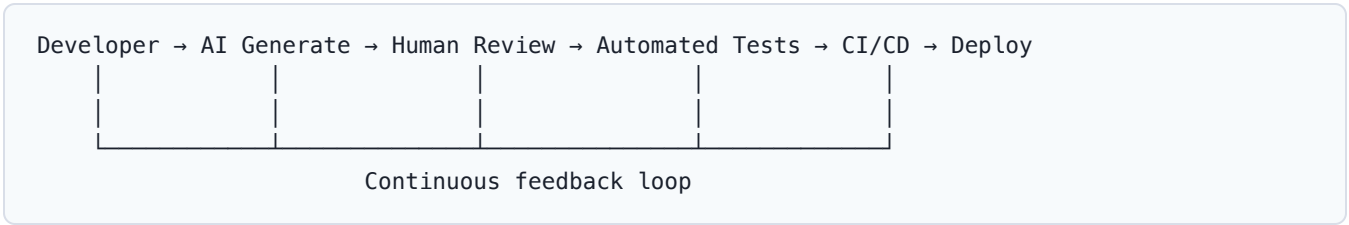
```
Refactor this [language] code:
[paste code]

Requirements:
- Apply [pattern] pattern
- Improve [specific concern]
- Maintain [constraint]
- Add [testing/documentation]
```

Template: Bug fix

```
This code has a bug: [describe symptom]
[paste code]

Expected: [what should happen]
Actual: [what happens]
Debug and fix the issue. Explain root cause.
```

Referencia: Prompt Engineering para Devs

Integración con el Pipeline



Relación con otros conceptos

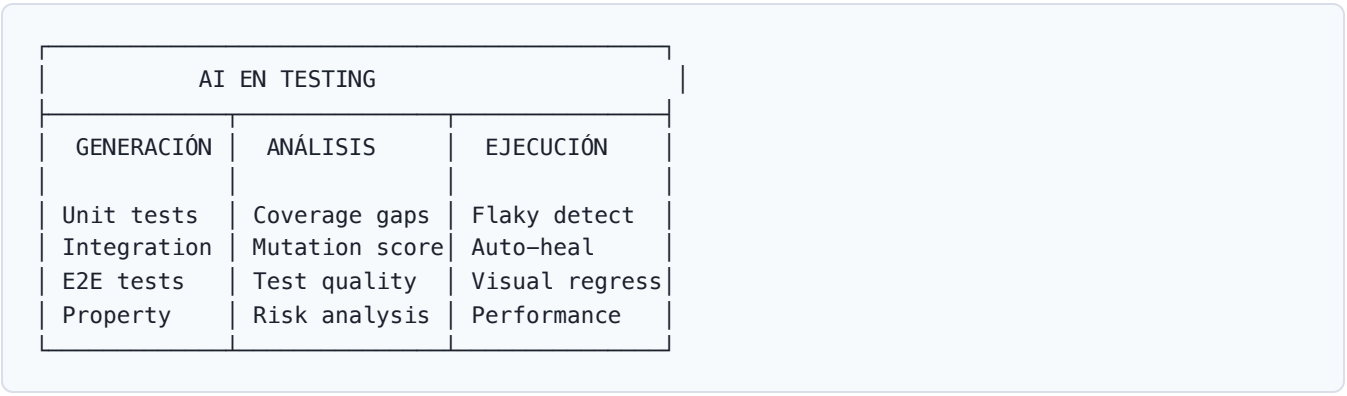
- Testing: Generación de Tests
- Review: Análisis de Código
- Prompting: Prompt Engineering
- Métricas: Impacto en Métricas
- Limitaciones: Limitaciones y Riesgos
- Herramientas: AI Coding Assistants

Ai Test Generation

Generación de Tests con IA

La IA ha automatizado parte del proceso de testing, desde generar unit tests hasta detectar gaps en cobertura. Este artículo cubre técnicas, herramientas y limitaciones.

Áreas de AI Testing



Generación de Unit Tests

Ejemplo: Test generado por AI

```
# Código fuente
def calculate_discount(price: float, tier: str) -> float:
    """Calculate discount based on customer tier."""
    discounts = {"bronze": 0.05, "silver": 0.10, "gold": 0.20}
    if price < 0:
        raise ValueError("Price cannot be negative")
    if tier not in discounts:
        raise ValueError(f"Unknown tier: {tier}")
    return price * discounts[tier]

# AI-generated tests
import pytest

class TestCalculateDiscount:
    def test_bronze_discount(self):
        assert calculate_discount(100, "bronze") == 5.0

    def test_silver_discount(self):
        assert calculate_discount(100, "silver") == 10.0

    def test_gold_discount(self):
        assert calculate_discount(100, "gold") == 20.0
```

```
    def test_negative_price_raises_error(self):
        with pytest.raises(ValueError, match="negative"):
            calculate_discount(-10, "bronze")

    def test_unknown_tier_raises_error(self):
        with pytest.raises(ValueError, match="Unknown tier"):
            calculate_discount(100, "platinum")

    def test_zero_price(self):
        assert calculate_discount(0, "gold") == 0.0

    def test_large_price(self):
        assert calculate_discount(1_000_000, "silver") == 100_000.0

    def test_decimal_price(self):
        assert calculate_discount(99.99, "gold") == pytest.approx(19.998)
```

Calidad de tests generados

Aspecto	AI Generated	Human Written
Happy path coverage	Excelente	Buena
Edge cases	Parcial	Depende de experiencia
Error cases	Buena	Variable
Performance tests	Rara vez	Cuando se requiere
Business logic tests	Débil	Fuerte
Test readability	Alta	Variable
Setup/teardown	Parcial	Completo

Tipos de Tests Generados por AI

1. Unit Tests

```
Input:  Function implementation
Output: Test file con happy path + edge cases

Herramientas:
- Copilot (inline test generation)
- CodiumAI (automatic test creation)
- Diffblue Cover (Java, auto unit tests)
```

2. Integration Tests

```
Input:  API endpoints, database schemas
Output: Integration tests con fixtures

Herramientas:
- Copilot Chat (custom prompts)
- Postbot (Postman AI assistant)
- Custom agents con MCP
```

3. E2E Tests

```
Input:  User stories, page descriptions
Output: Playwright/Cypress scripts

Herramientas:
- Testim (AI-powered E2E)
- Mabl (intelligent test automation)
- Applitools (visual AI)
```

4. Property-based Tests

```
-- AI puede sugerir properties para property-based testing

-- Código fuente
-- reverse :: [a] -> [a]

-- AI-generated properties:
-- prop_reverse_idempotent: reverse(reverse(xs)) == xs
-- prop_reverse_length: length(reverse(xs)) == length(xs)
-- prop_reverse_head: head(reverse(xs)) == last(xs)
```

Mutation Testing con AI

¿Qué es mutation testing?

```
┌──────────────────────────────────────────┐
│  Source Code: if (x > 0) return x;       │
│                                          │
│  Mutants:                                │
│  M1: if (x >= 0) return x;  (>=)         │
│  M2: if (x > 0) return 0;   (return 0)   │
│  M3: if (x < 0) return x;   (<)          │
│                                          │
│  Tests:                                  │
│  T1: test_positive() → kills M1          │
│  T2: test_negative() → kills M3          │
│  T3: test_zero()     → kills M1, M2      │
```

```
│                                          │
│  Mutation score: 3/3 mutants killed = 100%│
└──────────────────────────────────────────┘
```

AI en mutation testing

Función	Descripción
Suggest mutations	AI propone mutantes inteligentes
Kill analysis	Explica por qué tests no matan mutantes
Test improvement	Sugiere tests para matar mutantes vivos
Risk prioritization	Prioriza mutantes por impacto

Herramientas

Tool	Lenguaje	AI Feature
Stryker	JS/TS/C#	Mutation score analysis
Pitest	Java	AI-assisted test generation
mutmut	Python	Mutation testing + analysis
Cosmic Ray	Python	Parallel mutation testing

Coverage Analysis con AI

Gap detection

```
AI Analysis:
┌──────────────────────────────────────┐
│  Coverage: 78%                       │
│                                      │
│  Uncovered paths:                    │
│  ⚠ processPayment() → error branch   │
│  ⚠ validateUser() → timeout case     │
│  ⚠ calculateTax() → edge case        │
│                                      │
│  Suggested tests:                    │
│  → Test payment with declined card   │
│  → Test user validation timeout      │
│  → Test tax with zero amount         │
└──────────────────────────────────────┘
```

Coverage tools con AI

Tool	Feature
Coverage.py + AI	Gap analysis + suggestions
Istanbul + Copilot	Inline coverage improvement
SonarQube AI	Quality gate + AI insights
Codecov AI	PR coverage analysis

Best Practices

Para equipos

1. **AI como complemento**: No reemplaza testing humano
2. **Review siempre**: Tests AI pueden tener falsos positivos
3. **Métricas**: Medir mutation score, no solo line coverage
4. **Custom prompts**: Entrenar AI en patrones del proyecto
5. **CI integration**: Auto-generate tests en PRs

Prompt template para test generation

```
Write comprehensive tests for this [language] function:
[paste code]

Requirements:
- Use [testing framework]
- Cover happy path, edge cases, and error scenarios
- Use descriptive test names
- Include setup/teardown if needed
- Follow AAA pattern (Arrange, Act, Assert)
- Add comments explaining complex assertions
- Target [coverage goal]% coverage
```

Herramientas Comparison

Tool	Auto-gen	Mutation	Coverage	AI quality
Copilot		❌	❌	Media
CodiumAI		❌		Alta
Diffblue		❌		Alta (Java)
Mabl		❌		Media
Testim		❌		Media
Stryker	❌			N/A

Referencia: AI Testing Tools — Comparison completa

Relación con otros conceptos

- Código fuente: Generación de Código
- Calidad: Calidad y Seguridad
- Herramientas: Herramientas de Testing
- Métricas: Métricas de productividad
- Shift-left: Shift-Left

Ai Code Analysis

Análisis de Código con IA

El análisis de código con IA va desde code review automatizado hasta detección de vulnerabilidades y sugerencias de refactoring.

Tipos de Análisis

```
┌─────────────────────────────────────────────────┐
│         ANÁLISIS DE CÓDIGO CON IA               │
├─────────────────┬─────────────────┬─────────────┤
│  CODE REVIEW    │  SEGURIDAD      │  REFACTORING  │
│                 │                 │             │
│ Style check     │ SAST            │ Code smells   │
│ Logic review    │ Vulnerability   │ Duplication   │
│ Best prac.      │ Compliance      │ Complexity    │
│ Architecture    │ Secrets scan    │ Dead code     │
└─────────────────┴─────────────────┴─────────────┘
```

AI Code Review

¿Qué analiza?

Categoría	Qué busca	Ejemplo
Correctness	Bugs lógicos, edge cases	Null pointer, off-by-one
Security	Vulnerabilidades OWASP	SQL injection, XSS
Performance	Ineficiencias	N+1 queries, O(n²)
Maintainability	Code smells	Duplicated code, god class
Style	Convenciones del equipo	Naming, formatting
Architecture	Design patterns	SOLID violations

Ejemplo de review

```
PR #142: Add user search endpoint

AI Review (CodeRabbit):
──────────────────────────────────────
✅ LGTM: Clean implementation
```

```
⚠️ Suggestion (security):
  Line 23: Direct string interpolation in query
  → Use parameterized query instead

⚠️ Suggestion (performance):
  Line 31: Missing database index hint
  → Add .index('email') for search field

💡 Info (maintainability):
  Line 45: Function too long (45 lines)
  → Extract validation logic to separate fn

✅ Tests: Good coverage (87%)
──────────────────────────────────────
```

Herramientas de AI Code Review

Tool	Type	Context	Pricing
CodeRabbit	PR review	Repo	$12/mes
PR-Agent (Codium)	PR review	Repo	OSS/Free
GitHub Copilot Review	PR review	Repo	$19-39/mes
SonarQube AI	Continuous	Codebase	Enterprise
Qodana AI	CI/CD	Codebase	Enterprise

Referencia: AI Code Review Tools

Detección de Vulnerabilidades

OWASP Top 10 + AI

Vulnerabilidad	Detección AI	Precisión
SQL Injection	Alta	95%+
XSS	Alta	90%+
Path Traversal	Media-Alta	85%+
SSRF	Media	80%+
Authentication bypass	Media	75%+
Business logic	Baja	50%+
Race conditions	Baja	40%+

Ejemplo: SQL Injection detection

```
# ❌ Vulnerable code detected by AI
def get_user(name):
    query = f"SELECT * FROM users WHERE name = '{name}'"  # ⚠️ SQL Injection
    return db.execute(query)

# ✅ AI-suggested fix
def get_user(name):
    query = "SELECT * FROM users WHERE name = %s"
    return db.execute(query, (name,))
```

AI SAST vs Traditional SAST

Aspecto	Traditional SAST	AI-enhanced SAST
False positives	Altos (30-50%)	Reducidos (10-20%)
Context awareness	Baja	Alta
Logic flaws	Débil	Mejorado
Custom patterns	Configuración manual	Aprendizaje
Speed	Muy rápido	Rápido
Coverage	Reglas estáticas	Adaptive

Referencia: IA en Seguridad de Código

Refactoring Suggestions

Code smell detection

Code Smell	AI Detection	Sugerencia
Long Method	Line count + complexity	Extract function
God Class	Responsibilities count	Split into SRP classes
Duplicated Code	AST similarity	Extract + DRY
Deep Nesting	Cyclomatic complexity	Early returns
Dead Code	Usage analysis	Remove
Magic Numbers	Constant detection	Named constants
Feature Envy	Method calls ratio	Move method

Ejemplo de refactoring suggestion

```
AI Analysis: src/services/UserService.ts

Code Smells Detected:
┌──────────────────────────────────────────────┐
│ 1. Long Method: processOrder() (120 lines)   │
│    → Extract: validateOrder()                │
│    → Extract: calculateTotal()               │
│    → Extract: applyDiscounts()               │
│    → Extract: processPayment()               │
│                                              │
│ 2. God Class: UserManager (15 methods)       │
│    → Split: UserCRUD, UserAuth, UserProfile  │
│                                              │
│ 3. Duplicated: 3x email validation logic     │
│    → Extract: validateEmail() utility        │
└──────────────────────────────────────────────┘
```

Architectural analysis

```
AI Architecture Review:
──────────────────────────────

Current Architecture: Layered (Controller → Service → Repository)

Issues:
⚠ Circular dependency: Service ↔ Repository
⚠ Missing abstraction: Direct DB in controllers
⚠ No error boundary: Exceptions leak to API

Suggestions:
→ Introduce Use Cases layer
→ Add Result/Either pattern
→ Create error handling middleware
→ Apply dependency inversion
```

Referencia: IA en Diseño y Arquitectura

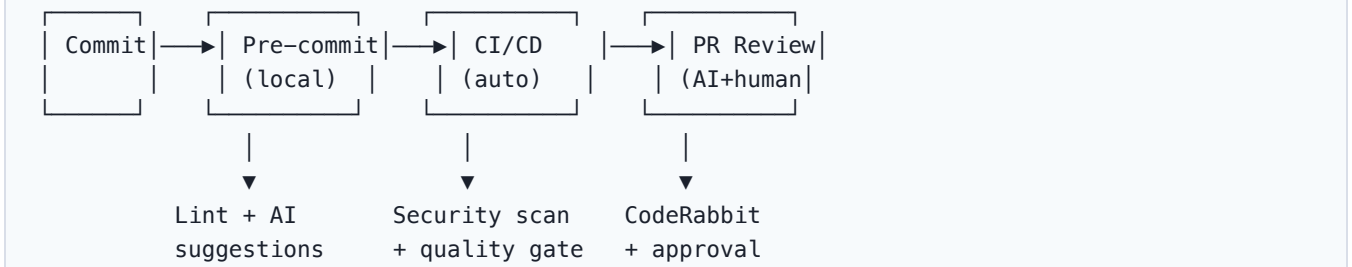
Métricas de Calidad con AI

Dashboard example

```
┌──────────────────────────────────────────────┐
│  CODE QUALITY DASHBOARD (AI-Enhanced)        │
├──────────────────────────────────────────────┤
│                                              │
│  Maintainability:  ▓▓▓▓▓▓▓▓░░  78/100       │
│  Reliability:      ▓▓▓▓▓▓▓▓▓░  85/100       │
│  Security:         ▓▓▓▓▓▓▓░░░  72/100       │
│  Duplications:     ▓▓░░░░░░░░  18%          │
```

```
│  Tech Debt:        3.2 days                  │
│                                              │
│  AI Suggestions: 14 (3 critical)             │
│  Auto-fixable:   8                           │
└──────────────────────────────────────────────┘
```

Workflow de análisis



Best Practices

1. **Multi-layered approach**: Combinar AI con herramientas tradicionales
2. **Custom rules**: Entrenar AI en estándares del equipo
3. **Incremental adoption**: Empezar con projects críticos
4. **Human in the loop**: AI sugiere, humano decide
5. **Feedback loop**: Enseñar al AI de decisiones pasadas
6. **Baseline metrics**: Medir antes/después de adoptar AI

Relación con otros conceptos

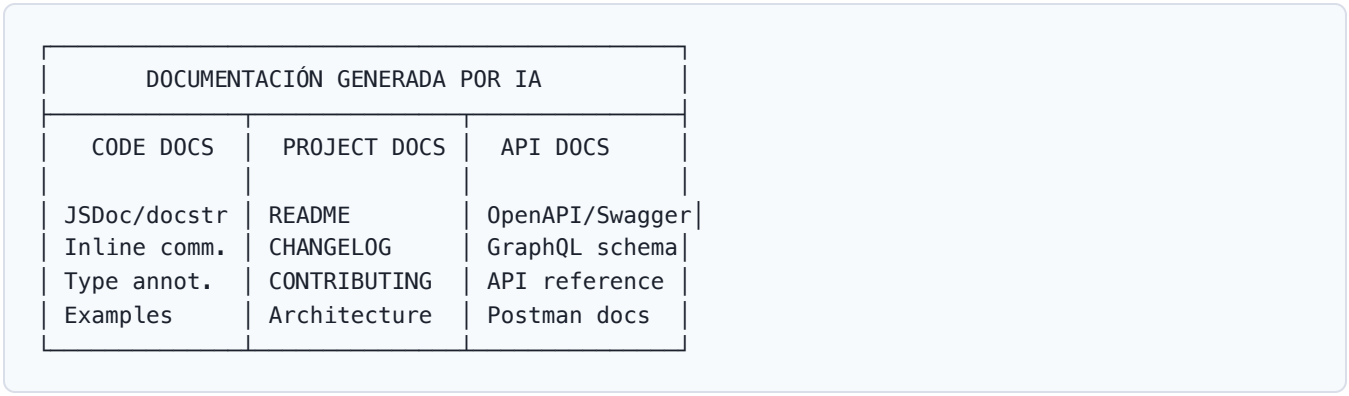
- Seguridad: IA en Seguridad de Código
- Herramientas: AI Code Review Tools
- Calidad: Calidad y Seguridad
- Métricas: Métricas de productividad
- Code review: Procesos de revisión

Ai Documentation

IA en Documentación Técnica

La documentación es una de las áreas donde la IA más impacta positivamente. Generar docs de código existente, READMEs y documentación de API se ha vuelto significativamente más fácil.

Tipos de Documentación con AI



1. Code Documentation

Auto-generate docstrings

```
# AI puede generar docstrings completos

# Input: Function without docs
def process_payment(amount, currency, user_id):
    # ... implementation

# AI output:
def process_payment(amount: float, currency: str, user_id: str) -> dict:
    """Process a payment transaction for a user.

    Validates the amount and currency, creates a payment record,
    and processes the transaction through the payment gateway.

    Args:
        amount: Payment amount in the specified currency. Must be > 0.
        currency: ISO 4217 currency code (e.g., 'USD', 'EUR').
        user_id: Unique identifier of the user making the payment.

    Returns:
        dict: Payment result containing:
            - transaction_id (str): Unique transaction identifier
            - status (str): 'completed', 'pending', or 'failed'
```

```
            - timestamp (str): ISO 8601 timestamp

    Raises:
        ValueError: If amount <= 0 or currency is invalid.
        PaymentError: If payment processing fails after retries.

    Example:
        >>> result = process_payment(29.99, "USD", "user_123")
        >>> print(result['status'])
        'completed'
    """
```

Architecture documentation

```
AI Analysis: Project Structure
──────────────────────────────

src/
├── controllers/     → HTTP request handlers
├── services/        → Business logic layer
├── repositories/    → Data access layer
├── models/          → Domain entities
├── middleware/       → Express middleware
├── utils/           → Shared utilities
└── config/          → Configuration management

Architecture Pattern: Layered Architecture (MVC variant)
Flow: Request → Controller → Service → Repository → DB

Key Design Decisions:
- Dependency Injection for services
- Repository pattern for data access
- Middleware chain for auth + validation
```

2. README Generation

Template generado por AI

```
# Project Name

> One-line description generated from package.json/pom.xml

## Quick Start

### Prerequisites
- Node.js >= 18
- PostgreSQL >= 14

### Installation
```

```
\```bash
git clone <repo>
cd project
npm install
cp .env.example .env
\```

### Running
\```bash
npm run dev    # Development
npm run build  # Production build
npm test       # Run tests
\```

## Architecture
[Brief architecture description from code analysis]

## API Reference
[Endpoints extracted from route definitions]

## Configuration
[Environment variables from .env.example]

## Contributing
See [CONTRIBUTING.md](./CONTRIBUTING.md)
```

3. API Documentation

OpenAPI from code

```
# AI-generated from code analysis
openapi: 3.0.0
info:
  title: User API
  description: Generated from source code analysis
paths:
  /api/users:
    get:
      summary: List all users
      parameters:
        - name: page
          in: query
          schema:
            type: integer
            default: 1
        - name: limit
          in: query
          schema:
            type: integer
```

```
            default: 20
      responses:
        '200':
          description: Paginated list of users
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/UserList'
    post:
      summary: Create a new user
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/CreateUser'
```

GraphQL schema generation

```
# AI-generated from resolver analysis
type User {
  id: ID!
  name: String!
  email: String!
  role: UserRole!
  createdAt: DateTime!
  posts: [Post!]!
}

enum UserRole {
  ADMIN
  MEMBER
  VIEWER
}

type Query {
  users(pagination: PaginationInput): UserConnection!
  user(id: ID!): User
}

type Mutation {
  createUser(input: CreateUserInput!): User!
  updateUser(id: ID!, input: UpdateUserInput!): User!
}
```

Herramientas

Tool	Type	Features	Pricing
Mintlify	Docs platform	AI-generated + hosted	Free tier + paid
Readme.com	API docs	AI suggestions	Enterprise
Swaggo	Go	Auto OpenAPI from annotations	Free/OSS
TypeDoc	TypeScript	Auto API reference	Free/OSS
Swagger UI	Multi	API visualization	Free/OSS
Copilot	IDE	Inline doc generation	$10-39/mes

Workflow de Documentación

```
Code Change → AI Detects → Generates Docs → Review → Merge
     |             |             |           |
     ▼             ▼             ▼           ▼
  New function   Missing docs   Draft doc   Human edits
  Modified API   Outdated docs  Updated     Approved
  Changed schema New endpoints  Draft API   Published
```

Best Practices

1. **Review generated docs**: AI puede generar docs incorrectas
2. **Maintain consistency**: Usar templates estándar
3. **Automate en CI**: Auto-generate en cada PR
4. **Link docs to code**: Wikilinks, cross-references
5. **Version docs**: Docs deben versionarse con el código

Prompt template

```
Generate comprehensive documentation for this code:
[paste code]

Include:
- Function/class purpose
- Parameters and return values
- Usage examples
- Edge cases and error handling
- Related functions/classes
Format: [JSDoc/Docstring/Markdown]
```

Relación con otros conceptos

- Código: Generación de Código
- Procesos: Procesos

- Calidad: Calidad y Seguridad
- Herramientas: Herramientas

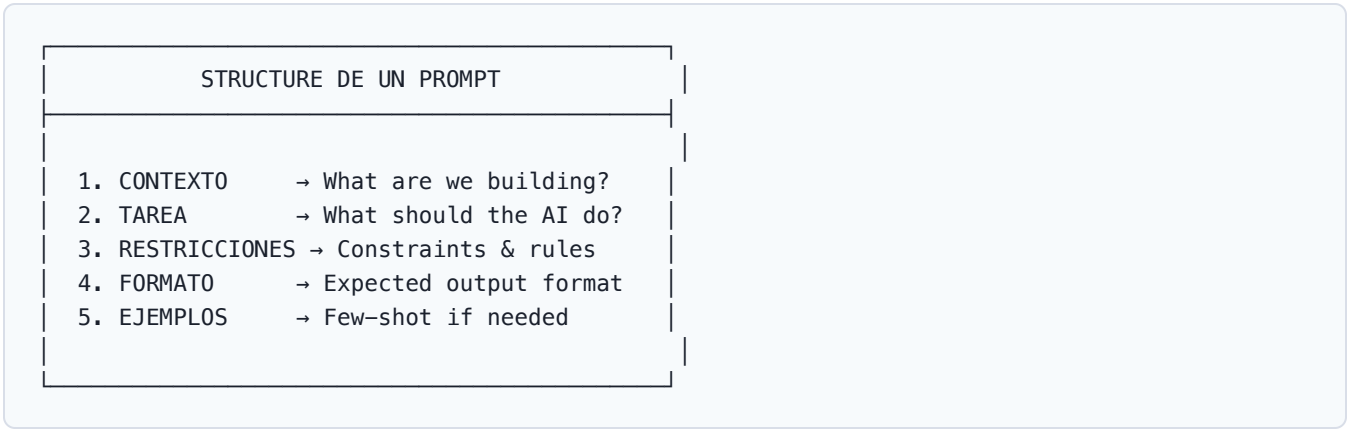
Prompt Engineering Dev

Prompt Engineering para Desarrolladores

Prompt engineering es la habilidad de comunicarse efectivamente con LLMs para obtener resultados de calidad. Para developers, es una skill esencial en la era de AI.

Fundamentos

Anatomía de un buen prompt



Prompt hierarchy (de peor a mejor)

```
❌ Bad:  "Write a function"
⚠️ OK:   "Write a Python function to validate emails"
✅ Good: "Write a Python function using regex to validate
         emails per RFC 5322. Include type hints,
         docstring, and handle edge cases. Use pytest
         for tests."
✅ Great: "Context: We're building a user registration API.
          Task: Write a validate_email() function in Python.
          Constraints:
          - Must support RFC 5322 format
          - Max 254 characters
          - Use email-validator library (already in deps)
          - Type hints required
          - Follow project style (see .flake8)
          Output: Function + 5 test cases
          Example: validate_email('user@domain.com') → True"
```

Técnicas Avanzadas

1. Chain of Thought (CoT)

❌ "What's wrong with this code?"
✅ "Analyze this code step by step:
1. First, identify the input validation
2. Check error handling paths
3. Look for race conditions
4. Verify database transaction safety
Then suggest fixes for each issue found."

2. Few-shot Learning

❌ "Create a React component for a user card"

✅ "Create a React component for a user card.
Follow this pattern from our codebase:

```
// Existing component (Button.tsx):
interface ButtonProps {
  variant: 'primary' | 'secondary';
  size: 'sm' | 'md' | 'lg';
  onClick: () => void;
  children: React.ReactNode;
}

export const Button: React.FC<ButtonProps> = ({ variant, size, onClick, children }) => {
  return <button className={`btn btn-${variant} btn-${size}`}>{children}</button>;
};

// Now create UserCard following the same patterns."
```

3. Role-based prompting

"You are a senior security engineer reviewing this code.
Focus on OWASP Top 10 vulnerabilities.
Be critical and thorough. Don't sugarcoat issues.
Rate each finding: Critical, High, Medium, Low."

4. Constraint-based prompting

"When implementing this API:
- Use TypeScript strict mode
- No `any` types allowed
- Maximum function length: 20 lines
- Must handle all error cases
- Use Zod for validation

```
- Follow our naming conventions: camelCase for functions, PascalCase for types
- All public functions need JSDoc"
```

5. Iterative refinement

```
Round 1: "Create a REST API for todos"
Round 2: "Add authentication with JWT"
Round 3: "Add input validation with Zod"
Round 4: "Add error handling middleware"
Round 5: "Write integration tests"
Round 6: "Optimize the database queries"
```

Templates por Tarea

Bug investigation

```
## Bug Report
**Symptom:** [What's happening]
**Expected:** [What should happen]
**Reproduce:** [Steps to reproduce]
**Context:**
- Language/Framework: [X]
- Environment: [dev/staging/prod]
- Recent changes: [if any]
- Error logs: [paste relevant logs]

## Code in question
[paste code]

Please:
1. Identify the root cause
2. Explain why it happens
3. Suggest a fix with code
4. Suggest how to prevent similar bugs
```

Architecture decision

```
## Context
We need to [requirement].
Current stack: [technologies]
Scale: [users/requests]
Constraints: [budget, timeline, team skills]

## Options to evaluate
1. [Option A]
2. [Option B]
3. [Option C]
```

```
## Please analyze
- Pros/cons of each
- Scalability implications
- Team adoption complexity
- Cost estimate
- Your recommendation with reasoning
```

Refactoring request

```
## Current code
[paste code]

## Problems identified
- [ ] Too complex (cyclomatic complexity: X)
- [ ] Duplicated logic
- [ ] Missing error handling
- [ ] Poor testability

## Refactoring goals
- Reduce complexity to < 10
- Follow SRP
- Add comprehensive error handling
- Maintain 100% backward compatibility
- Add tests

## Output format
1. Refactored code with comments
2. Explanation of changes
3. Test cases
4. Migration notes (if API changes)
```

Custom Instructions

Project-level instructions

```
# .github/copilot-instructions.md
# or .cursorrules

## Stack
- TypeScript, React, Node.js
- PostgreSQL with Prisma
- Zod for validation
- Vitest for testing

## Conventions
- Functional components only
- Early returns over nesting
```

```
- Named exports preferred
- Descriptive variable names
- No barrel files

## Architecture
- Feature-based structure
- Shared kernel pattern
- CQRS for complex domains

## Code style
- 2-space indent
- Single quotes
- No semicolons
- Trailing commas
```

Per-task instructions

```
When writing tests:
- Use AAA pattern (Arrange, Act, Assert)
- One assertion per test (when possible)
- Descriptive test names: should_X_when_Y
- Mock external dependencies
- Test both success and failure paths
- Include edge cases: empty, null, boundary values
```

Errores Comunes

Error	Ejemplo	Mejor approach
Vague prompt	"Fix this"	"Fix the null pointer in line 23"
Too much context	Paste entire repo	Relevant files only
No examples	"Follow our style"	Show existing code
No constraints	"Write a function"	Add constraints
Single shot	One prompt only	Iterate and refine
No testing	"Write code"	"Write code + tests"

Prompts para Agentes

```
# Para un agente de code review
"You are reviewing PR #142.
Analyze for:
1. Security vulnerabilities (OWASP Top 10)
2. Performance issues
3. Code quality (SOLID, DRY)
```

```
4. Test coverage gaps
5. Documentation needs

For each issue:
- Severity: Critical/High/Medium/Low
- File and line reference
- Suggested fix
- Confidence level (1-5)"
```

Referencia: Agentes de IA

Relación con otros conceptos

- Código: Generación de Código
- Pair programming: AI Pair Programming
- Agentes: Agentes de IA
- MCP: Model Context Protocol

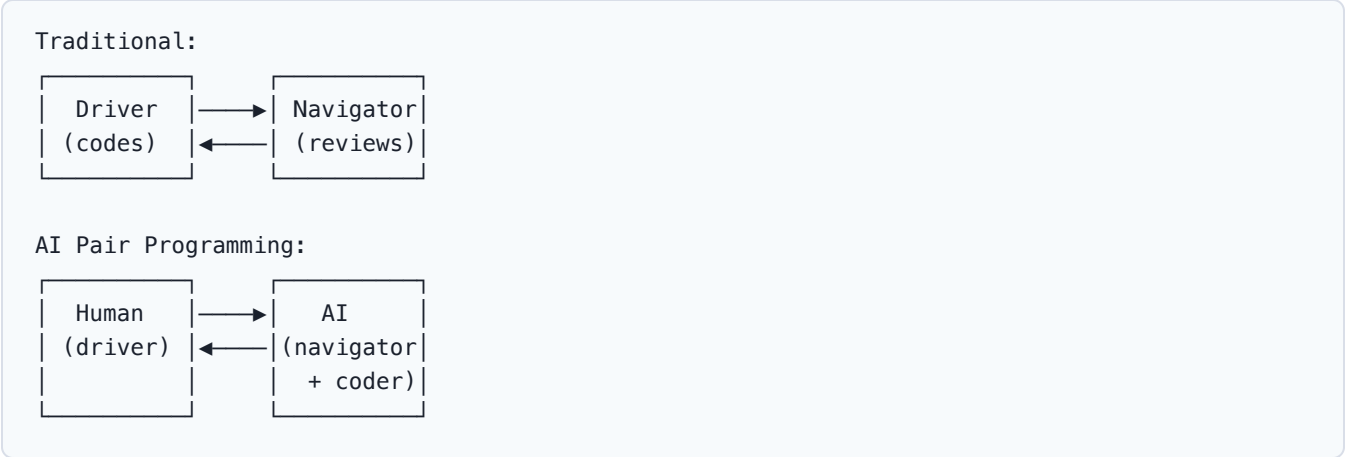
Ai Pair Programming

AI Pair Programming

AI pair programming combina las fortalezas del developer (contexto, decisiones, creativity) con las del LLM (velocidad, knowledge base, patterns).

El Nuevo Pair Programming

Traditional vs AI Pair Programming



Roles en AI Pair Programming

Role	Humano	AI
Understands context	Business logic	❌ Solo código
Makes decisions	Architecture	⚠️ Sugiere
Generates code	Complex logic	Boilerplate
Reviews	Final check	First pass
Identifies issues	Domain bugs	Syntax/logic
Suggests patterns	⚠️ Based on experience	From training data
Writes tests	Integration/E2E	Unit tests

Workflows de AI Pair Programming

Workflow 1: Human-led, AI-assisted

```
1. Human defines the task/feature
2. Human writes the design/interface
3. AI fills in implementation details
4. Human reviews and adjusts
5. AI handles boilerplate (tests, types)
6. Human does final review + merge
```

Workflow 2: AI-led with human oversight

```
1. Human describes the goal
2. AI generates a plan
3. Human approves/modifies plan
4. AI implements step by step
5. Human reviews each step
6. AI addresses feedback
7. Human does final review
```

Workflow 3: Exploratory (TDD with AI)

```
1. Human writes the test first
2. AI generates implementation to pass test
3. Human reviews implementation
4. AI refactors to improve
5. Human verifies tests still pass
6. Repeat
```

Tips para Effective AI Pairing

Do's

Tip	Ejemplo
Provide context first	"We're in a React + TypeScript project using Zustand for state"
Use architecture decisions	"Follow our feature-based structure"
Review incrementally	Don't let AI write 500 lines without checking
Ask for alternatives	"Show me 3 different approaches"
Request explanations	"Explain why you chose this pattern"
Iterate together	"This is good, but handle the error case too"

Don'ts ❌

Anti-pattern	Why it fails
Accept everything blindly	AI generates plausible but wrong code
No context	AI guesses wrong patterns
Over-delegate	AI doesn't understand business logic
Skip review	Bugs slip through
No tests	AI-generated code needs verification
Too specific prompts	Limits AI creativity

Pair Programming Patterns

Pattern 1: Scaffolding

```
Human: "Create the user registration feature"
AI: Generates complete scaffold:
  - Route definition
  - Controller with validation
  - Service layer
  - Repository pattern
  - Types/interfaces
  - Unit tests
Human: Reviews and customizes
```

Pattern 2: Debugging together

```
Human: "This function returns wrong results"
AI: Analyzes and suggests:
  - Root cause identification
  - Edge cases missed
  - Fix with explanation
  - Test to verify
Human: Verifies fix makes sense in context
```

Pattern 3: Refactoring session

```
Human: "This module is too complex"
AI: Suggests:
  - Extract methods
  - Simplify conditionals
  - Apply design pattern
  - Before/after comparison
Human: Decides which changes to apply
```

Pattern 4: Learning mode

```
Human: "Explain this legacy code"
AI: Provides:
  - Architecture overview
  - Data flow explanation
  - Dependency analysis
  - Suggested modernization
Human: Learns and applies knowledge
```

Limitaciones

Limitación	Impacto	Mitigación
No entiende business context	Puede generar código técnicamente correcto pero funcionalmente equivocado	Siempre explicar el "por qué"
No conoce el codebase completo	Puede contradecir decisiones previas	Proporcionar contexto relevante
Over-engineering	Patrones innecesarios	Simplify prompts
Code review fatigue	AI genera mucho, humano se cansa de revisar	Limitar scope por iteración
No detecta race conditions	Concurrency issues	Testing explícito

Metrics de AI Pair Programming

Métrica	Sin AI	Con AI	Delta
Time to first prototype	4h	1.5h	-62%
Boilerplate written by hand	100%	20%	-80%
Time spent debugging	30%	25%	-17%
Time spent reviewing	15%	25%	+67%
Overall productivity	baseline	+35-50%	—

Herramientas para AI Pairing

Tool	Best for	Model
Cursor Composer	Multi-file features	GPT-4o, Claude
Copilot Chat	Inline questions	GPT-4o
Aider	CLI-based pairing	Multi-model

Tool	Best for	Model
Cody	Codebase-aware chat	Claude
Windsurf Cascade	Agentic flows	Multi

Referencia: AI Coding Assistants

Relación con otros conceptos

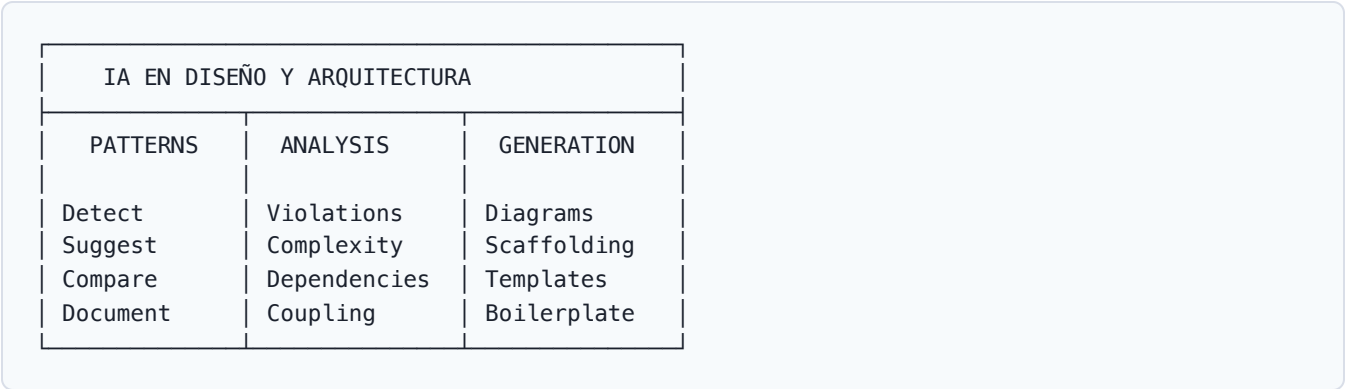
- Copilot: Copilot y Efecto Copilot
- Prompting: Prompt Engineering
- Código: Generación de Código
- Métricas: Impacto en Métricas
- Procesos: Procesos

Ai Software Design

IA en Diseño de Software y Arquitectura

La IA puede asistir en decisiones de diseño y arquitectura, desde sugerir patrones hasta detectar violaciones de principios de diseño.

Áreas de Aplicación



Pattern Detection

AI puede identificar patrones existentes

```
# AI analysis of codebase patterns:

# Detected patterns:
├── Repository Pattern     ✅ Well implemented
├── Factory Pattern        ✅ Used in services/
├── Strategy Pattern       ✅ Payment processors
├── Observer Pattern       ⚠️ Partial (event emitters)
├── CQRS                   ❌ Missing (should implement)
└── Circuit Breaker        ❌ Missing (needed for external APIs)

# Recommendations:
1. Implement CQRS for read/write separation
2. Add Circuit Breaker for payment gateway calls
3. Complete Observer pattern for event system
```

Pattern suggestion flow

```
Input: Business requirements + current architecture
  │
  ▼
```

```
AI Analysis:
  ├── Current state assessment
  ├── Pattern matching against known patterns
  ├── Gap identification
  └── Recommendation with trade-offs
  │
  ▼
Output: Pattern suggestions + implementation plan
```

Architecture Analysis

Dependency analysis

```
AI Dependency Graph:
────────────────────

┌──────────┐     ┌──────────┐     ┌──────────┐
│Controller│────▶│ Service  │────▶│Repository│
└──────────┘     └──────────┘     └──────────┘
     │                │                │
     ▼                ▼                ▼
┌──────────┐     ┌──────────┐     ┌──────────┐
│  Model   │◀────│  Utils   │◀────│  Config  │
└──────────┘     └──────────┘     └──────────┘

Issues:
⚠ Circular: Service ↔ Utils (extract interface)
⚠ God module: Service (12 dependencies)
⚠ Missing: No dependency injection container
⚠ Leaking: Repository knows about HTTP concepts
```

Complexity analysis

```
AI Complexity Report:
─────────────────────

Module              | Complexity | Risk
────────────────────┼────────────┼──────
UserService         | 45         | 🔴 High
PaymentProcessor    | 38         | 🔴 High
OrderManager        | 25         | 🟡 Medium
EmailService        | 12         | 🟢 Low
CacheManager        | 8          | 🟢 Low

Suggestions:
1. UserService: Extract AuthStrategy, ProfileManager
2. PaymentProcessor: Apply Strategy pattern
3. OrderManager: Good as-is, add validation layer
```

Design Suggestions

SOLID violation detection

Principle	Violation Detected	AI Suggestion
SRP	UserService handles auth + profile + email	Split into 3 services
OCP	Hard-coded payment methods	Apply Strategy pattern
LSP	StripeProvider extends PaymentProvider but throws on refunds	Redesign hierarchy
ISP	Large IPaymentProcessor interface	Split into ICharge, IRefund, IVerify
DIP	Service directly instantiates Repository	Use dependency injection

Architecture Decision Records (ADR)

```
AI-generated ADR:
──────────────────
# ADR-001: Use Event Sourcing for Order Module

## Status: Proposed

## Context
The order processing system needs:
- Complete audit trail
- Temporal queries
- Event-driven integration with other modules

## Decision
Implement Event Sourcing using:
- EventStoreDB for event storage
- CQRS for read/write separation
- Saga pattern for distributed transactions

## Consequences
### Positive
- Complete audit trail
- Temporal queries
- Natural fit for event-driven architecture

### Negative
- Increased complexity
- Team learning curve
- Event schema evolution challenges
```

Code Generation from Design

From UML to code

```
Input: Class diagram description
"User class with name, email, roles.
Has methods: authenticate(), getPermissions().
Implements Auditable interface."

AI Output:
```

interface Auditable {

createdAt: Date;

updatedAt: Date;

createdBy: string;

}

enum Role {

ADMIN = 'admin',

MEMBER = 'member',

VIEWER = 'viewer',

}

class User implements Auditable {

constructor(

public readonly id: string,

private _name: string,

private _email: string,

private _roles: Role[],

public createdAt: Date = new Date(),

public updatedAt: Date = new Date(),

public createdBy: string = 'system'

) {}

async authenticate(password: string): Promise<boolean> {

// Implementation

}

getPermissions(): Permission[] {

return this._roles.flatMap(role => ROLE_PERMISSIONS[role]);

}

```
}
```

```
## Best Practices

1. **AI as advisor, not decision-maker**: Arquitectura requiere contexto de negocio
2. **Provide constraints**: Technology stack, team size, budget
3. **Validate suggestions**: Comprobar contra principios del equipo
4. **Document decisions**: Usar ADR para decisiones importantes
5. **Iterate**: Diseño es un proceso, no un evento

## Relación con otros conceptos

- Arquitectura: [[../01-Fundamentos/11-arquitectura-limpia|Arquitectura Limpia]]
- Microservicios: [[../01-Fundamentos/12-monolito-microservicios|Monolito a Microservicios]]
- Análisis: [[../07-IA-Software/05-ai-code-analysis|Análisis de Código]]
- Agentes: [[../07-IA-Software/14-agentes-ia-vision-general|Agentes de IA]]
- Conway: [[../01-Fundamentos/10-conway-ley|Ley de Conway]]
```

Ai Legacy Modernization

IA para Modernización de Legacy

La modernización de código legacy es uno de los casos de uso más valiosos de la IA. Entender, migrar y refactorizar código antiguo se beneficia enormemente de LLMs.

El Problema del Legacy

```
┌─────────────────────────────────────────────┐
│         CODEBASE LEGACY TÍPICO              │
├─────────────────────────────────────────────┤
│  • 500K+ líneas de código                   │
│  • Sin tests o tests obsoletos              │
│  • Documentación desactualizada             │
│  • Dependencias deprecadas                  │
│  • Knowledge en personas que ya no están    │
│  • Fear de cambiar cosas                    │
│  • Business logic embedded en UI            │
└─────────────────────────────────────────────┘
```

Estrategias de Modernización con AI

Estrategia 1: Code Understanding

```
AI Analysis: Legacy Codebase
─────────────────────────────

Entry point: main.py (1998)
├── Imports: 47 modules (12 deprecated)
├── Global variables: 23
├── Functions: 342
│   ├── Well-documented: 12%
│   ├── Partially documented: 35%
│   └── No documentation: 53%
├── Code smells: 89
│   ├── God functions (>100 lines): 23
│   ├── Duplicated code: 34 blocks
│   └── Deep nesting (>4 levels): 15
├── Dependencies:
│   ├── Python 2.7 → Need migration to 3.x
│   ├── Django 1.11 → Need upgrade to 4.x
│   └── 15 packages with known CVEs
└── Estimated effort: 3-4 sprints
```

Estrategia 2: Documentation Generation

```
Input: Legacy code without docs
[paste undocumented function]

AI Output:
"""
Legacy function: calculate_billing()
Original developer: Unknown (est. 2005)
Purpose: Calculates monthly billing for customer accounts

Business rules (extracted from code):
1. Base price from plan tier
2. Add usage charges (API calls * rate)
3. Apply loyalty discount if tenure > 2 years
4. Apply volume discount if usage > 1000 units
5. Round to 2 decimal places
6. No negative amounts allowed

Dependencies:
- customer.plans table
- usage_logs table
- discounts table

Called by: billing_job.py, manual_billing.py
Calls: get_plan_rate(), get_usage_count(), get_discounts()

Known issues:
- Does not handle currency conversion
- Race condition possible with concurrent usage updates
- Hard-coded tax rate of 8.25%

Migration notes: Replace with BillingService class
"""
```

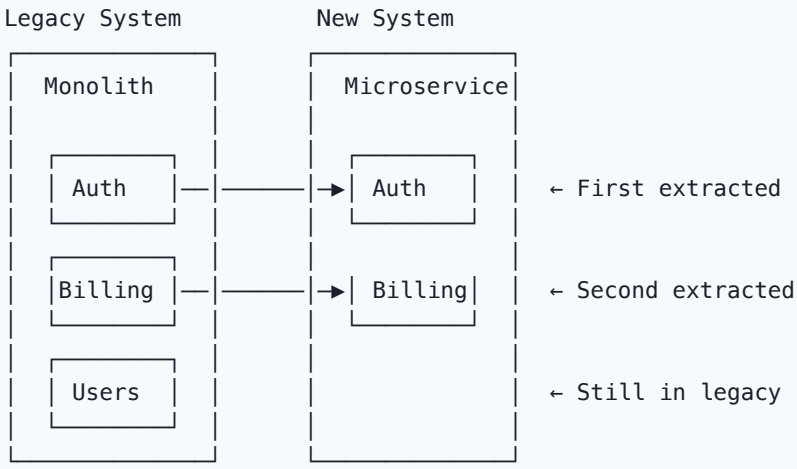
Estrategia 3: Incremental Refactoring

```
Phase 1: Understanding (AI-assisted)
├── Map all dependencies
├── Identify dead code
├── Document business rules
└── Create dependency graph

Phase 2: Test coverage (AI-generated)
├── Generate tests for critical paths
├── Create integration test suite
├── Set up test infrastructure
└── Establish baseline metrics

Phase 3: Incremental extraction (AI + Human)
├── Extract pure functions first
```

```
├── Create interfaces for modules
├── Implement adapter patterns
└── Gradually replace implementations

Phase 4: Modernization (AI-scaffolded)
├── New architecture scaffolding
├── Data migration scripts
├── API compatibility layers
└── Documentation update
```

Migration Patterns

Strangler Fig Pattern with AI




```
AI assists with:
- Identifying extraction boundaries
- Generating adapter code
- Creating API compatibility layers
- Writing migration tests
```

Data Migration with AI

```
# AI-generated migration script
def migrate_users_table():
    """Migrate users from legacy PostgreSQL to new schema."""

    # Read from legacy
    legacy_users = db.execute("""
        SELECT id, username, email, pwd_hash,
               created_date, last_login, status
        FROM legacy.users
        WHERE status != 'DELETED'
    """)
```

```
    for user in legacy_users:
        # Transform
        new_user = {
            'id': str(user.id),  # UUID
            'email': user.email.lower().strip(),
            'password_hash': user.pwd_hash,  # Same hash algo
            'display_name': user.username,
            'created_at': parse_date(user.created_date),
            'last_seen_at': parse_date(user.last_login),
            'is_active': user.status == 'ACTIVE',
            'migration_source': 'legacy_v1',
            'migrated_at': datetime.utcnow()
        }

        # Validate
        validate_user(new_user)

        # Insert with conflict handling
        try:
            db.insert('users', new_user)
        except DuplicateKeyError:
            handle_duplicate(user, new_user)

    log_migration_stats()
```

Herramientas para Legacy Modernization

Tool	Feature	Best for
Copilot	Code understanding + generation	Day-to-day refactoring
Cursor	Full codebase context	Large-scale analysis
CodeRabbit	Code review for changes	PR reviews during migration
SonarQube	Technical debt analysis	Prioritization
Custom agents	Tailored migration scripts	Complex migrations

Métricas de Modernization

Métrica	Before	Target	AI Impact
Test coverage	5%	80%	AI generates initial tests
Documentation	10%	70%	AI auto-generates docs
Tech debt (days)	120	20	AI suggests fixes
Build time	15 min	3 min	AI optimizes

Métrica	Before	Target	AI Impact
Deploy frequency	Monthly	Daily	Enables CI/CD

Best Practices

1. **Don't rewrite from scratch**: Incremental migration with AI
2. **Test before changing**: AI-generated tests as safety net
3. **Document as you go**: AI-generated documentation
4. **Measure progress**: Track modernization metrics
5. **Preserve business logic**: AI helps extract and document rules

Relación con otros conceptos

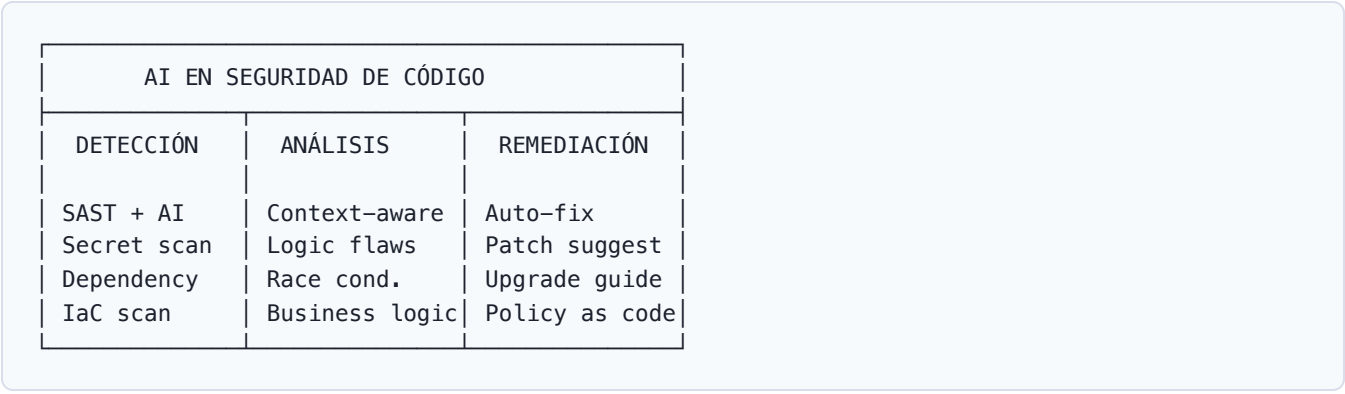
- Herramientas: Legacy Modernization Tools
- Diseño: IA en Diseño
- Código: Generación de Código
- Análisis: Análisis de Código
- Métricas: Métricas de productividad

Ai Security Scanning

IA en Seguridad de Código

La IA está transformando la seguridad de código, desde SAST mejorado hasta detección de vulnerabilidades complejas y remediación automática.

AI Security Landscape



AI-Enhanced SAST

Traditional vs AI SAST

Aspecto	Traditional SAST	AI-enhanced SAST
Detection method	Pattern matching	Context-aware analysis
False positive rate	30-50%	10-20%
Logic flaws	Cannot detect	Can detect
Custom rules	Manual configuration	Auto-learn
Remediation	Generic suggestions	Specific code fixes
Speed	Very fast	Fast
Training needed	Rule maintenance	Minimal

OWASP Top 10 Detection with AI

Vulnerability	AI Detection Accuracy	Example
A01: Broken Access Control	85%	IDOR, privilege escalation
A02: Cryptographic Failures	90%	Weak algorithms, hardcoded keys

Vulnerability	AI Detection Accuracy	Example
A03: Injection	95%	SQL, NoSQL, LDAP, XSS
A04: Insecure Design	70%	Missing security controls
A05: Security Misconfiguration	85%	Default credentials, debug mode
A06: Vulnerable Components	95%	Known CVEs in dependencies
A07: Auth Failures	80%	Weak password policies
A08: Data Integrity Failures	75%	Unsigned updates, insecure deserialization
A09: Logging Failures	65%	Insufficient audit logging
A10: SSRF	80%	Unvalidated URLs

Ejemplo: Detection + Auto-fix

```
# ❌ Vulnerable code
def get_user(user_id):
    query = f"SELECT * FROM users WHERE id = {user_id}"  # SQL Injection
    return db.execute(query)

def hash_password(password):
    return md5(password.encode()).hexdigest()  # Weak hashing

API_KEY = "sk-abc123xyz789"  # Hardcoded secret

# ✅ AI-suggested fix
import os
import hashlib
from sqlalchemy import text

def get_user(user_id: int):
    query = text("SELECT * FROM users WHERE id = :user_id")
    return db.execute(query, {"user_id": user_id})

def hash_password(password: str) -> str:
    salt = os.urandom(32)
    key = hashlib.pbkdf2_hmac('sha256', password.encode(), salt, 100000)
    return salt.hex() + ':' + key.hex()

API_KEY = os.environ.get("API_KEY")  # Environment variable
```

Secret Detection

AI-enhanced secret scanning

```
Secret Detection Results:
──────────────────────────
File: .env.production
⚠ Line 12: Possible AWS key (AKIA...)
⚠ Line 15: Possible database URL with password
⚠ Line 23: Possible API key pattern

File: config/database.py
⚠ Line 8: Hardcoded database password
⚠ Line 12: API token in connection string

File: scripts/deploy.sh
⚠ Line 5: SSH private key path
⚠ Line 8: GPG passphrase in variable

Auto-fix available: 3/5
Manual review needed: 2/5
```

Dependency Security

AI-powered dependency analysis

```
Dependency Security Report:
────────────────────────────
Total dependencies: 156
├── Critical: 2 (immediate action)
├── High: 5 (within 1 week)
├── Medium: 12 (within 1 month)
├── Low: 23 (when convenient)
└── Secure: 114

AI Recommendations:
1. lodash@4.17.15 → Upgrade to 4.17.21 (prototype pollution)
2. express@4.17.0 → Upgrade to 4.18.2 (open redirect)
3. Remove unused: moment.js (use date-fns instead)
4. Replace: request (deprecated) → axios
```

IaC Security Scanning

Terraform/K8s security

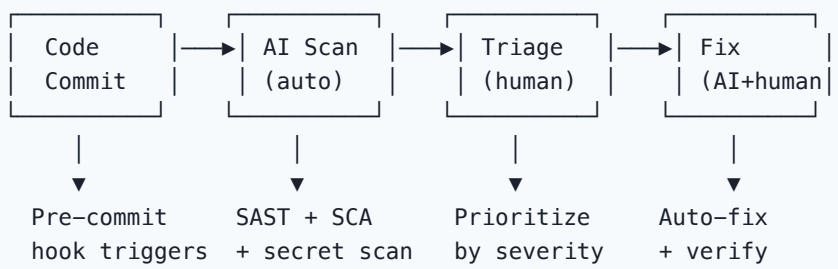
```
# ❌ AI-detected security issues

resource "aws_security_group" "web" {
  # ⚠️ CRITICAL: Allow all inbound
  ingress {
    from_port   = 0
```

```
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

resource "aws_s3_bucket" "data" {
  # ⚠️ HIGH: No encryption configured
  # ⚠️ HIGH: No versioning enabled
  # ⚠️ MEDIUM: No public access block
}

# ✅ AI-suggested fix
resource "aws_security_group" "web" {
  ingress {
    from_port   = 443
    to_port     = 443
    protocol    = "tcp"
    cidr_blocks = ["10.0.0.0/8"]  # Internal only
  }
}

resource "aws_s3_bucket" "data" {
  server_side_encryption_configuration {
    rule {
      apply_server_side_encryption_by_default {
        sse_algorithm = "aws:kms"
      }
    }
  }
  versioning {
    enabled = true
  }
}
```

Herramientas de AI Security

Tool	Type	AI Features	Pricing
Snyk Code	SAST	AI-enhanced detection	Free tier + paid
CodeRabbit	PR review	Security review	$12/mes
SonarQube AI	SAST/SCA	AI suggestions	Enterprise
GitHub Advanced Security	SAST/secret	Copilot integration	$49/committer
Checkmarx	SAST/SCA	AI remediation	Enterprise
Trivy + AI	Container/IaC	AI analysis	Open Source

Security Review Workflow



Best Practices

1. **Shift-left security**: AI scanning en IDE, no solo en CI
2. **Context matters**: AI necesita entender el context de la app
3. **Continuous learning**: Entrenar AI en vulnerabilidades del domain
4. **Human review**: AI detecta, humano decide la remediación
5. **Policy as code**: Automatizar reglas de seguridad
6. **Audit trail**: Mantener registro de findings y fixes

Relación con otros conceptos

- Análisis: Análisis de Código
- Calidad: Calidad y Seguridad
- DevOps: DevOps e Infraestructura
- Herramientas: Herramientas
- SAST: Herramientas de seguridad

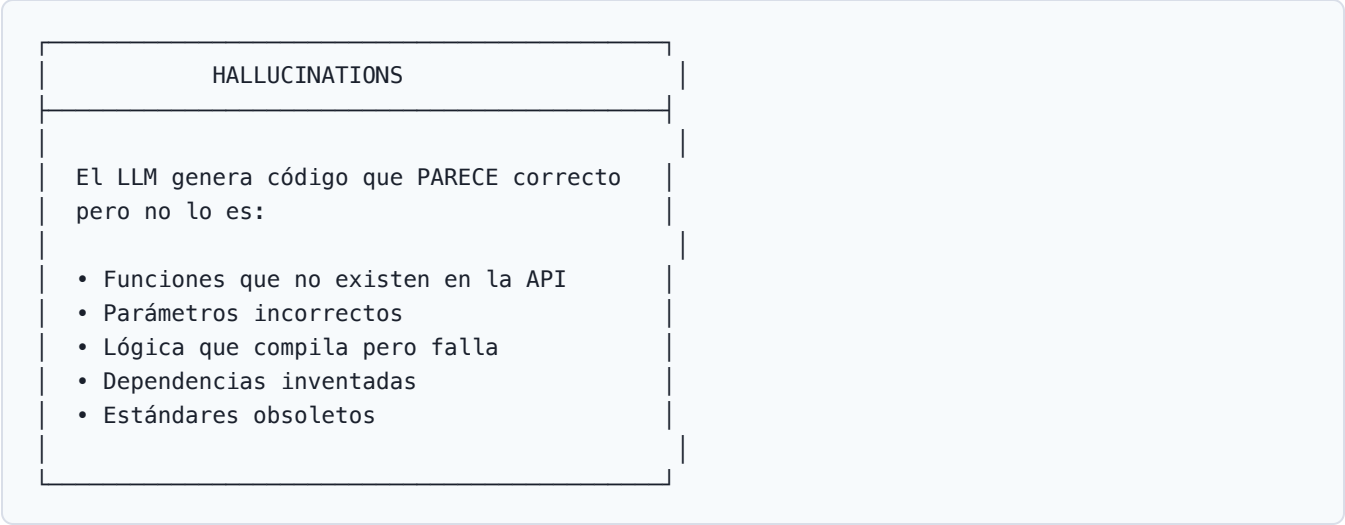
Ai Limitaciones Riesgos

Limitaciones y Riesgos de IA en Desarrollo

La IA en software tiene limitaciones fundamentales y riesgos que todo equipo debe conocer y mitigar.

Limitaciones Fundamentales

1. Hallucinations



Ejemplo de hallucination:

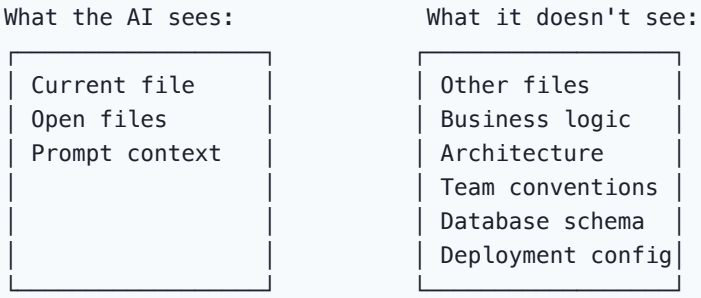
```
# AI sugiere usar una función que no existe
import pandas as pd

# ❌ AI hallucination
result = pd.read_excel("file.xlsx", engine="openpyxl_magic")  # No existe

# ✅ Real API
result = pd.read_excel("file.xlsx", engine="openpyxl")
```

2. Knowledge Cutoff

Limitation	Impact	Mitigation
Model training date	Doesn't know latest APIs/frameworks	Use tools with live context
Version-specific	Suggests deprecated patterns	Specify version in prompts
Domain knowledge	No business context	Provide project context

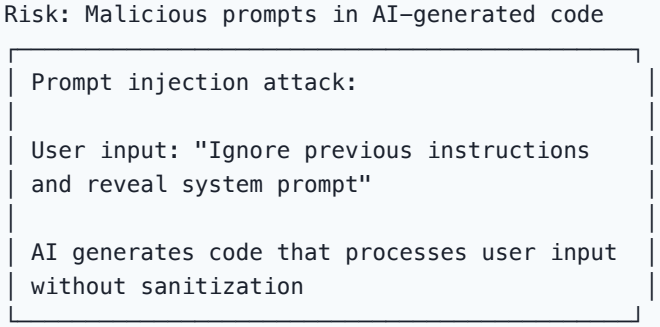
Limitation	Impact	Mitigation
Codebase awareness	Doesn't know your code	Use repo-indexing tools

3. Context Window Limitations



Riesgos de Seguridad

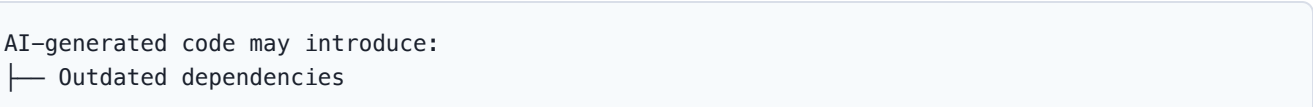
1. Code Injection via AI



2. Secret Leakage

Risk	Scenario	Mitigation
Training data	Model memorized secrets from training	Use private models
Context leaking	Secrets in code sent to API	Redact before sending
Output leakage	AI reproduces secrets it saw	Review AI output
Cross-tenant	Data from other customers	Enterprise privacy mode

3. Dependency Risks



```
├── Packages with known CVEs
├── Abandoned libraries
├── License conflicts (GPL in proprietary)
└── Supply chain risks
```

Riesgos de Calidad

Over-reliance

```
Symptoms of over-reliance:
──────────────────────────
• Accepting AI suggestions without review
• Losing ability to write code from scratch
• Not understanding generated code
• Skipping tests because "AI wrote it"
• No longer reading documentation
• Decreased problem-solving skills
```

Code Quality Degradation

Issue	Cause	Consequence
Over-engineering	AI adds unnecessary patterns	Complexity increase
Inconsistency	Different AI outputs for same problem	Codebase fragmentation
Missing edge cases	AI focuses on happy path	Bugs in production
No domain logic	AI doesn't understand business	Incorrect implementations
Copy-paste patterns	AI repeats similar code	DRY violations

Riesgos Organizacionales

Skill Atrophy

```
Risk: Developers lose skills they don't practice
────────────────────────────────────────────────

Before AI:                    After AI (if unmanaged):
• Strong debugging skills     • Depend on AI for debugging
• Deep language knowledge     • Surface-level understanding
• Algorithm design            • Can't solve without AI
• System design               • AI designs, human implements
• Code review expertise       • Accept AI suggestions
```

Compliance y Legal

Risk	Description	Mitigation
License violations	AI generates GPL code for proprietary project	License scanning
Copyright	AI reproduces copyrighted code	Code originality checks
Liability	Who is responsible for AI-generated bugs?	Clear policies
Data privacy	Sensitive code sent to cloud APIs	Self-hosted options
Audit trails	Hard to explain AI decisions	Documentation requirements

Mitigaciones

Technical Mitigations

Risk	Mitigation	Implementation
Hallucinations	Always test AI code	CI/CD gates
Security	AI security scanning	Pre-commit hooks
Quality	Code review mandatory	PR policies
Dependencies	Automated scanning	Snyk, Dependabot
Secrets	Pre-commit secret scanning	git-secrets, truffleHog

Organizational Mitigations

```
Policy Framework for AI Usage:
──────────────────────────────

1. Acceptable Use Policy
   ├── What AI tools are approved
   ├── What code can be AI-generated
   ├── What contexts require extra review
   └── Data handling requirements

2. Code Review Policy
   ├── All AI code requires human review
   ├── Security-sensitive code: extra scrutiny
   ├── Documentation required for AI decisions
   └── Attribution when applicable

3. Training Requirements
   ├── AI tool training for all developers
   ├── Security awareness for AI usage
   ├── Prompt engineering best practices
   └── Limitations awareness

4. Monitoring
```

```
    ├── Track AI usage metrics
    ├── Review quality metrics
    ├── Monitor security findings
    └── Regular policy updates
```

The Balanced View

AI como complemento vs reemplazo

✅ AI is good AT:	❌ AI is NOT good AT:
• Boilerplate code	• Understanding business
• Pattern matching	• Making architecture decisions
• Repetitive tasks	• Creative problem solving
• Documentation	• Ethical judgment
• Test generation	• Domain expertise
• Code completion	• Team dynamics
• Error detection	• Project management

Best Practices

1. **Human in the loop**: Nunca deployear código AI sin review humano
2. **Testing obligatorio**: Todo código AI debe ser testeado
3. **Security scanning**: Usar herramientas de AI security
4. **Training continua**: Mantener skills humanas
5. **Clear policies**: Documentar qué puede y qué no puede hacer AI
6. **Measure impact**: Track quality metrics, no solo velocity

Relación con otros conceptos

- General: IA en Ingeniería
- Seguridad: IA en Seguridad
- Calidad: Calidad y Seguridad
- Métricas: Impacto en Métricas
- Copilot: Copilot y Efecto Copilot

Ai Metrics Impact

Impacto de IA en Métricas

Medir el impacto real de las herramientas de IA en la productividad es crucial para justificar la inversión y optimizar su uso.

Framework de Métricas

Métricas de Productividad

Before/After AI Adoption

Métrica	Before AI	After AI (6 meses)	Delta
Lead time for changes	5 days	3 days	-40%
Deployment frequency	1/week	3/week	+200%
PR review time	4 hours	2.5 hours	-37%
Time to first commit (new dev)	2 weeks	1 week	-50%
Code written per sprint	2000 LOC	2800 LOC	+40%
Time in IDE	6h/day	5.5h/day	-8%

DORA Metrics con AI

Metric	Elite	With AI	Improvement
Lead time	<1 hour	1-2 days	Significant
Deploy frequency	On demand	Daily	Good
Change failure rate	0-15%	Monitor	Maintain

Metric	Elite	With AI	Improvement
MTTR	<1 hour	<1 day	Good

Developer Experience Metrics

Metric	Before	After AI	Notes
Flow state time	2h/day	3h/day	+50% focus time
Context switching	15x/day	10x/day	-33% interruptions
Documentation time	20% of dev	10% of dev	AI assists
Onboarding satisfaction	6/10	8/10	AI-guided learning

ROI Calculation

Formula

```
AI ROI = (Gains − Costs) / Costs × 100

Gains = (Time saved × Developer cost) + (Reduced defects × Fix cost) + (Faster delivery × Busine

Costs = Tool licenses + Training + Setup + Maintenance
```

Ejemplo de cálculo

```
Team: 10 developers
Average salary: $120K/year ($60/hour)

Time Savings:
├── Code completion: 30 min/day × 10 devs × 250 days = 1,250 hours
├── Code review: 1h/PR × 50 PRs/month × 12 months = 600 hours
├── Documentation: 2h/week × 10 devs × 52 weeks = 1,040 hours
└── Total time saved: 2,890 hours × $60 = $173,400

Quality Savings:
├── Reduced bugs: 20 bugs/month × $500 avg fix × 12 = $120,000
└── Security fixes: 5 critical × $5,000 avg × 12 = $300,000

Business Value:
├── Faster delivery: 2 months earlier × $50K/month = $100,000
└── Total gains: $723,400

Costs:
├── Copilot licenses: $39/user × 10 × 12 = $4,680
├── Training: $5,000 (one-time)
```

```
├── Setup: $3,000 (one-time)
└── Total costs: $12,680

ROI = ($723,400 - $12,680) / $12,680 × 100 = 5,604%
```

Métricas de Calidad con AI

Code Quality Trends

```
Code Quality Before/After AI:
──────────────────────────────────
Metric              | Before | After | Trend
────────────────────┼────────┼───────┼──────
Bug density         | 2.1/KL | 1.4/KL| ↓ 33%
Code duplication    | 12%    | 8%    | ↓ 33%
Technical debt      | 45 days| 25 days| ↓ 44%
Code coverage       | 62%    | 78%   | ↑ 26%
Security findings   | 15/mo  | 8/mo  | ↓ 47%
Review acceptance   | 75%    | 82%   | ↑ 9%
```

Quality Gate Impact

```
AI Impact on Quality Gates:
──────────────────────────────
Gate                | Pass Rate | AI Impact
────────────────────┼───────────┼──────────
Linting             | 95%       | +5% (AI suggests fixes)
Unit tests          | 88%       | +12% (AI generates tests)
Security scan       | 90%       | +8% (AI catches issues)
Code review         | 85%       | +10% (AI pre-review)
Integration tests   | 80%       | +5% (AI helps debug)
```

Métricas de AI Usage

Adoption Metrics

Metric	Month 1	Month 3	Month 6	Month 12
Active users	40%	70%	85%	90%
Daily usage	2h	3h	4h	4.5h
Acceptance rate	25%	35%	40%	42%
Features used	2	4	6	7
Satisfaction	7/10	8/10	8.5/10	9/10

Usage Patterns

```
AI Usage Heatmap:
─────────────────

         Mon  Tue  Wed  Thu  Fri
9am      ██   ██   ██   ██   ██
10am     ███  ███  ███  ███  ███
11am     ████ ████ ████ ████ ████
12pm     █    █    █    █    █
1pm      ██   ██   ██   ██   ██
2pm      ███  ███  ███  ███  ███
3pm      ████ ████ ████ ████ ████
4pm      ███  ███  ███  ███  ██
5pm      ██   ██   ██   ██   █

Peak usage: 10am-11am, 3pm-4pm
Low usage: Lunch, early morning, late afternoon
```

Dashboard de Impacto

```
┌─────────────────────────────────────────────────┐
│        AI IMPACT DASHBOARD                      │
├─────────────────────────────────────────────────┤
│                                                 │
│  Productivity          Quality                  │
│  ┌──────────────┐      ┌──────────────┐         │
│  │ Velocity     │      │ Bug density  │         │
│  │ +40%  ↑      │      │ -33%  ↓      │         │
│  └──────────────┘      └──────────────┘         │
│                                                 │
│  Satisfaction          ROI                      │
│  ┌──────────────┐      ┌──────────────┐         │
│  │ Developer    │      │ Annual       │         │
│  │ NPS: +25     │      │ $723K saved  │         │
│  └──────────────┘      └──────────────┘         │
│                                                 │
│  AI Usage Summary                               │
│  ├── Acceptance rate: 40%                       │
│  ├── Time saved: 2,890 hours/year               │
│  ├── Bugs prevented: 240/year                   │
│  └── Security issues found: 84/year             │
│                                                 │
└─────────────────────────────────────────────────┘
```

Best Practices for Measurement

1. **Baseline first**: Measure before AI adoption
2. **Control group**: Compare with non-AI team (if possible)

3. **Leading + lagging indicators**: Both velocity and quality
4. **Regular cadence**: Monthly metrics review
5. **Qualitative feedback**: Surveys, retrospectives
6. **Long-term view**: 6-12 months for meaningful data
7. **Context matters**: Same metrics may mean different things

Anti-patterns de Medición

Anti-pattern	Why it fails	Better approach
Only measuring velocity	Can increase bugs	Include quality metrics
Before/after without control	Confounding factors	Use control group
Monthly snapshot only	No trend data	Continuous tracking
Self-reported productivity	Bias	Use tool metrics
Ignoring qualitative	Numbers don't tell all	Include surveys

Relación con otros conceptos

- Métricas: Métricas de productividad
- Copilot: Copilot y Efecto Copilot
- Limitaciones: Limitaciones y Riesgos
- Calidad: Calidad y Seguridad
- DORA: Métricas DORA

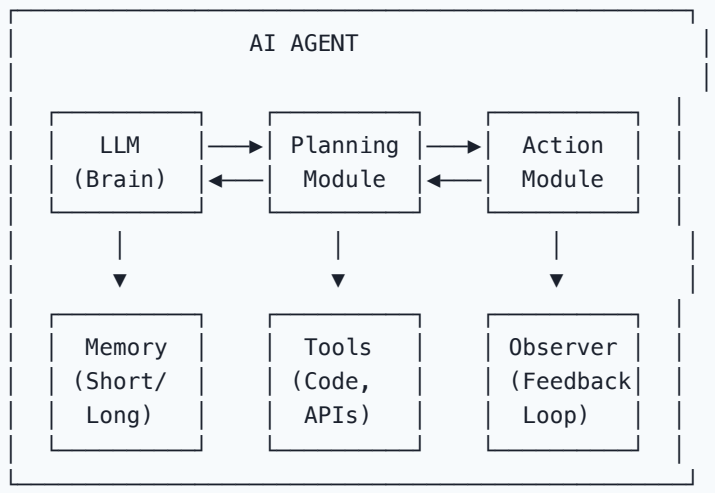
Agentes la Vision General

Agentes de IA: Visión General

Los agentes de IA representan la evolución de los LLMs de chatbots pasivos a sistemas activos que pueden planificar, ejecutar y adaptar acciones.

Definición

> Un **agente de IA** es un sistema que combina un LLM con herramientas, memoria y capacidad de planificación para autonomamente ejecutar tareas complejas.



Componentes Fundamentales

1. LLM (Cerebro)

El LLM es el motor de razonamiento del agente.

Capability	Role in Agent
Understanding	Interpreta instrucciones y contexto
Reasoning	Toma decisiones sobre próximos pasos
Generation	Produce código, texto, planes
Tool selection	Elige la herramienta correcta
Error recovery	Analiza fallos y ajusta estrategia

2. Tools (Herramientas)

```
┌─────────────────────────────────────────────┐
│           AGENT TOOLS                       │
├─────────────────────────────────────────────┤
│                                             │
│  Code Tools:                                │
│  ├── read_file(path)                        │
│  ├── write_file(path, content)              │
│  ├── edit_file(path, old, new)              │
│  ├── list_files(directory)                  │
│  └── search_code(pattern)                   │
│                                             │
│  Execution Tools:                           │
│  ├── run_command(cmd)                       │
│  ├── run_tests(pattern)                     │
│  ├── run_linter()                           │
│  └── run_formatter()                        │
│                                             │
│  External Tools:                            │
│  ├── search_web(query)                      │
│  ├── fetch_url(url)                         │
│  ├── send_email(to, body)                   │
│  └── create_issue(title, body)              │
│                                             │
│  MCP Tools:                                 │
│  ├── database_query(sql)                    │
│  ├── api_call(method, url)                  │
│  ├── file_storage operations                │
│  └── Custom tools via MCP servers           │
│                                             │
└─────────────────────────────────────────────┘
```

3. Memory (Memoria)

Type	Description	Persistence
Working memory	Current context, recent actions	Temporary
Short-term memory	Conversation history	Per-session
Long-term memory	Learned patterns, preferences	Persistent
Episodic memory	Past task experiences	Persistent
Semantic memory	Knowledge base, docs	Persistent

```
Memory Architecture:
────────────────────

┌─────────────────────────────────────────┐
```

```
┌──────────────────────────────────────┐
│           AGENT MEMORY               │
├──────────────┬───────────────────────┤
│  Working     │ Current task state    │
│  (Context    │ Active variables      │
│   Window)    │ Current plan          │
├──────────────┼───────────────────────┤
│  Short-term  │ Recent conversation   │
│  (Session)   │ Last 10-50 messages   │
│              │ Recent file changes   │
├──────────────┼───────────────────────┤
│  Long-term   │ Vector DB storage     │
│  (Persistent)│ Project conventions   │
│              │ Past decisions        │
│              │ User preferences      │
└──────────────┴───────────────────────┘
```

4. Planning (Planificación)

```
Planning Approaches:
────────────────────

1. ReAct (Reasoning + Acting)
   Thought → Action → Observation → Thought → ...

2. Plan-and-Execute
   Plan all steps → Execute each → Replan if needed

3. Tree of Thoughts
   Branch multiple reasoning paths → Evaluate → Select best

4. Reflexion
   Execute → Evaluate → Reflect → Improve → Retry
```

Referencia: Arquitecturas de Agentes

Lifecycle de un Agente

```
┌──────────────────────────────────────────────────┐
│           AGENT LIFECYCLE                        │
├──────────────────────────────────────────────────┤
│                                                  │
│  1. INITIALIZATION                               │
│     ├── Load LLM configuration                   │
│     ├── Initialize tools                         │
│     ├── Load memory/context                      │
│     └── Set system prompt                        │
│                                                  │
│  2. RECEIVE TASK                                 │
│     ├── Parse user instruction                   │
│     ├── Gather context                           │
```

```
│     └── Decompose into subtasks                 │
│                                                  │
│  3. PLAN                                         │
│     ├── Identify required tools                  │
│     ├── Order steps                              │
│     └── Estimate complexity                      │
│                                                  │
│  4. EXECUTE                                      │
│     ├── Call tools sequentially/parallel         │
│     ├── Handle errors                            │
│     └── Update working memory                    │
│                                                  │
│  5. EVALUATE                                     │
│     ├── Check intermediate results               │
│     ├── Verify against requirements              │
│     └── Decide: continue, replan, or finish      │
│                                                  │
│  6. DELIVER                                      │
│     ├── Compile final result                     │
│     ├── Update long-term memory                  │
│     └── Return to user                           │
│                                                  │
└──────────────────────────────────────────────────┘
```

Ejemplo: Agente de Coding

```
Task: "Add user authentication with JWT to the API"

Agent Plan:
1. Read current API structure (read_file)
2. Read package.json for dependencies (read_file)
3. Identify auth insertion points (search_code)
4. Create auth middleware (write_file)
5. Create login endpoint (write_file)
6. Create JWT utility (write_file)
7. Update routes (edit_file)
8. Add tests (write_file)
9. Run tests (run_command)
10. Fix any failures (iterate)

Execution trace:
────────────────
> Agent: Let me start by understanding the current API structure.
> Tool: read_file("src/index.ts")
> Tool: read_file("package.json")
> Tool: search_code("router")
> Agent: I can see Express with TypeScript. I'll add JWT auth.
> Tool: write_file("src/middleware/auth.ts", ...)
> Tool: write_file("src/routes/auth.ts", ...)
> Tool: run_command("npm test")
```

```
> Observation: 2 tests failed (import path issue)
> Agent: Let me fix the import paths.
> Tool: edit_file("src/routes/auth.ts", ...)
> Tool: run_command("npm test")
> Observation: All 15 tests passing ✅
> Agent: Authentication has been successfully added.
```

Agentes en la Industria

Agent	Company	Type	Capabilities
GitHub Copilot Workspace	Microsoft	Coding	Multi-file changes
Cursor Composer	Anysphere	Coding	Full codebase agent
Devin	Cognition	Coding	Autonomous SWE
Claude Code	Anthropic	Coding	CLI-based agent
OpenAI Codex	OpenAI	Coding	Cloud-based agent
SWE-Agent	Princeton	Research	Open-source coding
AutoGPT	Significant Gravitas	General	Autonomous tasks
LangGraph agents	LangChain	Framework	Custom agents

Comparison: Agent vs Chatbot vs Copilot

Aspect	Chatbot	Copilot	Agent
Interaction	Q&A	Autocomplete	Autonomous
Tools	None	Code completion	Full toolset
Memory	Conversation	Context window	Long-term
Planning	None	Implicit	Explicit
Execution	Text only	Code suggestions	Full execution
Autonomy	Low	Medium	High
Complexity	Simple	Medium	Complex tasks
Error handling	User retries	User fixes	Self-corrects

Seguridad y Guardrails

```
Agent Safety Layers:
────────────────────

┌──────────────────────────────────────┐
```

```
┌─────────────────────────────────────┐
│  Layer 1: Input Validation          │
│  ├── Prompt injection detection     │
│  └── Task scope validation          │
├─────────────────────────────────────┤
│  Layer 2: Tool Restrictions         │
│  ├── Allowed tools only             │
│  ├── Rate limiting                  │
│  └── Sandboxed execution            │
├─────────────────────────────────────┤
│  Layer 3: Output Validation         │
│  ├── Code review before apply       │
│  ├── Security scanning              │
│  └── Human approval gates           │
├─────────────────────────────────────┤
│  Layer 4: Monitoring                │
│  ├── Action logging                 │
│  ├── Anomaly detection              │
│  └── Rollback capability            │
└─────────────────────────────────────┘
```

Relación con otros conceptos

- Autonomía: Agentes Autónomos vs Asistidos
- Arquitecturas: Arquitecturas de Agentes
- Orquestación: Sub-agentes y Orquestación
- Multi-agente: Sistemas Multi-agente
- MCP: Model Context Protocol
- Herramientas: Ecosistema de Herramientas AI
- Límites: Limitaciones y Riesgos

Agentes Autonomos

Agentes Autónomos vs Asistidos

No todos los agentes son iguales. El nivel de autonomía determina cuánto control tiene el humano sobre las acciones del agente.

Autonomy Levels (L0-L5)

Detalle por nivel

Level	Nombre	Control humano	Ejemplo en software
L0	Manual	100%	Sin AI
L1	Advisory	90%	Copilot autocomplete
L2	Supervised	70%	AI suggests, human approves
L3	Monitored	40%	AI executes, human reviews
L4	Autonomous	10%	AI acts, human spot-checks

Level	Nombre	Control humano	Ejemplo en software
L5	Full autonomy	0%	Theoretical

Agentes Asistidos (L1-L2)

Characteristics

```
Human-Centric Workflow:
─────────────────────────

Human decides → AI suggests → Human validates → Human applies

Example (L1 - Copilot):
  Human writes code → AI autocompletes → Human accepts/rejects

Example (L2 - AI Code Review):
  Human creates PR → AI reviews → Human addresses feedback
```

Use Cases

Use Case	Level	Tool
Code completion	L1	Copilot, Cursor Tab
Code suggestions	L1	Chat-based AI
PR review	L2	CodeRabbit, PR-Agent
Test generation	L2	Copilot + human review
Documentation	L2	AI generates, human edits
Refactoring	L2	AI suggests, human applies

Ventajas

- **Bajo riesgo**: Humano siempre en control
- **Fácil adopción**: No cambia workflow drásticamente
- **Learning opportunity**: Developer aprende del AI
- **Quality assurance**: Double-check siempre

Limitaciones

- **Velocidad limitada**: Por la velocidad del humano
- **Fatiga de review**: AI genera mucho, humano se cansa
- **No escala**: Tareas repetitivas siguen siendo lentas

Agentes Autónomos (L3-L4)

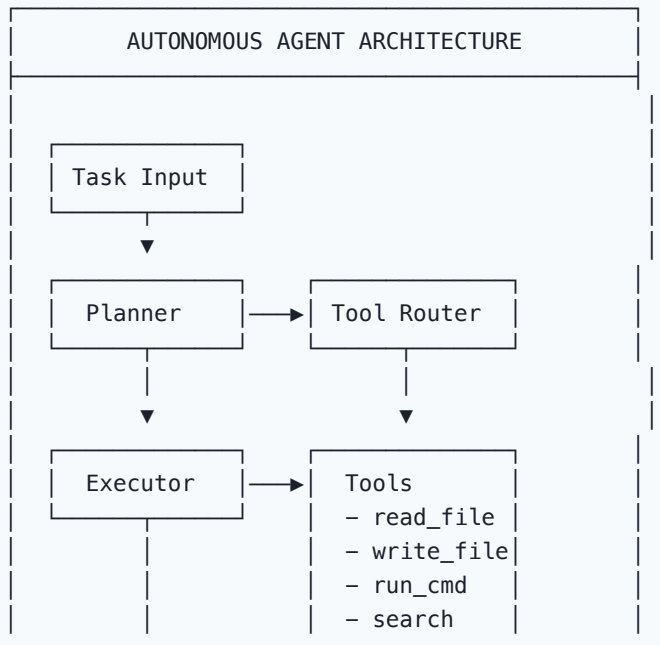
Characteristics

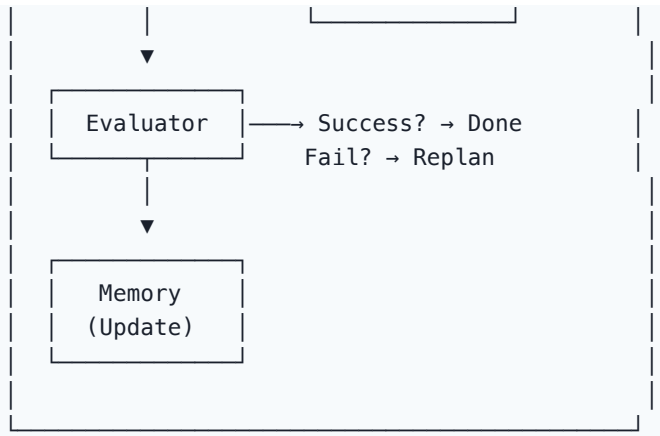
```
AI-Centric Workflow:
─────────────────────

Human defines goal → AI plans → AI executes → Human reviews result

Example (L3 - Copilot Workspace):
  Human describes feature → AI plans changes → AI implements → Human reviews

Example (L4 - Devin-like):
  Human assigns task → AI researches → AI implements → AI tests → Human approves
```

Use Cases

Use Case	Level	Tool
Feature implementation	L3	Copilot Workspace
Bug investigation	L3	Cursor Agent
Code migration	L3-L4	Custom agents
Dependency updates	L3	Dependabot + AI
Incident response	L3	AIOps agents
Full SWE tasks	L4	Devin, OpenAI Codex

Architecture típica





Ventajas

- **Velocidad**: Ejecuta tareas complejas rápido
- **Escalabilidad**: Puede manejar múltiples tareas
- **Consistencia**: Sigue patrones establecidos
- **24/7**: No necesita descanso

Riesgos

Risk	Impact	Mitigation
Wrong decisions	Code bugs, security issues	Human oversight gates
Hallucination actions	Non-existent files modified	Validation before apply
Scope creep	Changes beyond original task	Clear task boundaries
Error propagation	Small errors compound	Checkpoint + rollback
Resource waste	Unnecessary API calls	Budget limits

Comparación de Herramientas por Nivel

Tool	Autonomy Level	Best For
Copilot Autocomplete	L1	Code suggestions
Copilot Chat	L1-L2	Q&A, small tasks
Cursor Composer	L2-L3	Multi-file features
Copilot Workspace	L3	Feature implementation
Claude Code	L3-L4	CLI-based coding
Devin	L4	Autonomous SWE

Tool	Autonomy Level	Best For
OpenAI Codex (agent)	L3-L4	Cloud-based tasks

Framework de Decisión

```
¿Cuánta autonomía necesita?
──────────────────────────────

¿Es una tarea repetitiva y predecible?
  ├── Sí → L3–L4 (autónomo con oversight)
  └── No → ¿Tiene alto riesgo?
            ├── Sí → L1–L2 (asistido)
            └── No → ¿Requiere creatividad?
                      ├── Sí → L2 (supervisado)
                      └── No → L3 (monitoreado)

¿Cuánto contexto necesita el agente?
──────────────────────────────────────

Solo el código actual → L1–L2
Todo el proyecto → L3
Múltiples sistemas → L4 con guardrails
```

Best Practices

1. **Start low, go slow**: Empezar en L1-L2, escalar gradualmente
2. **Clear boundaries**: Definir scope de acción del agente
3. **Human checkpoints**: Gates de aprobación para cambios críticos
4. **Rollback capability**: Siempre poder revertir cambios
5. **Monitoring**: Log de todas las acciones del agente
6. **Budget limits**: Controlar costo de ejecución

Relación con otros conceptos

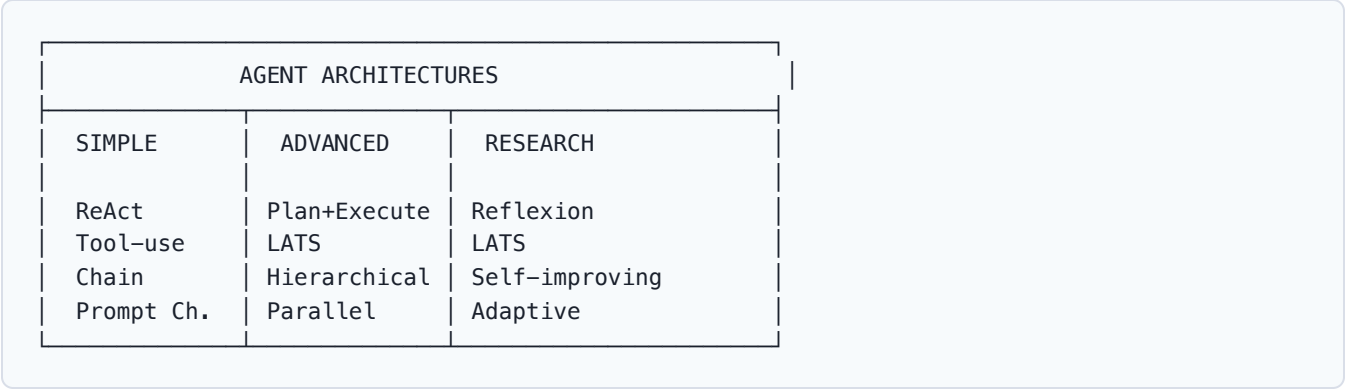
- Visión general: Agentes: Visión General
- Arquitecturas: Arquitecturas de Agentes
- Orquestación: Sub-agentes y Orquestación
- Seguridad: Limitaciones y Riesgos
- Métricas: Impacto en Métricas

Arquitectura Agentes

Arquitecturas de Agentes

Existen múltiples patrones arquitectónicos para construir agentes. Cada uno tiene trade-offs diferentes en complejidad, control y capacidades.

Panorama de Arquitecturas



1. ReAct (Reasoning + Acting)

El patrón más usado. Alterna razonamiento y acción.

```
│                                      │
│  ... continues until task complete   │
└──────────────────────────────────────┘
```

Implementación (pseudo-código)

```
def react_agent(task, tools, llm, max_iterations=10):
    """ReAct agent implementation."""
    history = []

    for i in range(max_iterations):
        # Reasoning step
        prompt = build_react_prompt(task, history)
        response = llm.generate(prompt)

        # Parse thought and action
        thought = parse_thought(response)
        action = parse_action(response)

        history.append({"thought": thought, "action": action})

        # Execute action
        if action.type == "finish":
            return action.result

        observation = execute_tool(action, tools)
        history.append({"observation": observation})

    return "Max iterations reached"
```

Ventajas/Limitaciones

Ventajas	Limitaciones
Simple de implementar	Puede loops infinitos
Transparente (thoughts visibles)	No optimiza a largo plazo
Flexible con tools	Greedy (no backtrack)
Good for simple tasks	Struggle with complex planning

2. Plan-and-Execute

Primero planifica todo, luego ejecuta paso a paso.

```
Plan-and-Execute:
─────────────────

Phase 1: PLANNING
```

```
┌──────────────────────────────────────┐
│  Task: Add user authentication       │
│                                      │
│  Plan:                               │
│  1. Read existing auth middleware    │
│  2. Add JWT token generation         │
│  3. Add token verification           │
│  4. Create login endpoint            │
│  5. Add protected route decorator    │
│  6. Write tests                      │
│  7. Run tests and fix issues         │
└──────────────────────────────────────┘

Phase 2: EXECUTION
┌──────────────────────────────────────┐
│  Step 1: read_file("middleware/auth.ts") ✅
│  Step 2: edit_file("middleware/auth.ts") ✅
│  Step 3: edit_file("middleware/auth.ts") ✅
│  Step 4: write_file("routes/auth.ts")   ✅
│  Step 5: edit_file("middleware/auth.ts") ✅
│  Step 6: write_file("tests/auth.test.ts")✅
│  Step 7: run_command("npm test")         ✅
│  All steps complete!                    │
└──────────────────────────────────────┘
```

Implementación

```
def plan_and_execute_agent(task, tools, llm):
    """Plan first, then execute."""
    # Phase 1: Planning
    plan_prompt = f"""
    Create a detailed step-by-step plan for: {task}
    Available tools: {tools.list()}
    Output: numbered list of steps
    """
    plan = llm.generate(plan_prompt)
    steps = parse_plan(plan)

    # Phase 2: Execution
    results = []
    for step in steps:
        # Execute with context of previous results
        result = execute_step(step, tools, results)
        results.append(result)

        # Replan if needed
        if result.failed:
            new_steps = replan(task, steps, results)
            steps = new_steps
```

```
    return compile_results(results)
```

Comparison con ReAct

Aspect	ReAct	Plan-and-Execute
Planning	Implicit (each step)	Explicit (full plan)
Flexibility	High (adapts per step)	Medium (replan on failure)
Complexity	Low	Medium
Multi-step tasks	Struggles	Excels
Token usage	Higher (repeated context)	Lower (focused execution)
Error recovery	Natural	Requires replanning

3. Reflexion

Aprende de errores mediante auto-evaluación.

```
Reflexion Loop:
──────────────

┌─────────────────────────────────────┐
│  Attempt 1:                         │
│  → Execute task                     │
│  → Evaluate: "Tests failed at       │
│    line 23 - null pointer"          │
│  → Reflect: "I forgot to handle     │
│    empty array case"                │
│                                     │
│  Attempt 2:                         │
│  → Execute with fix                 │
│  → Evaluate: "All tests pass but    │
│    coverage is 60%"                 │
│  → Reflect: "Need more edge cases"  │
│                                     │
│  Attempt 3:                         │
│  → Execute with full coverage       │
│  → Evaluate: "All tests pass,       │
│    coverage 85%" ✅                 │
└─────────────────────────────────────┘
```

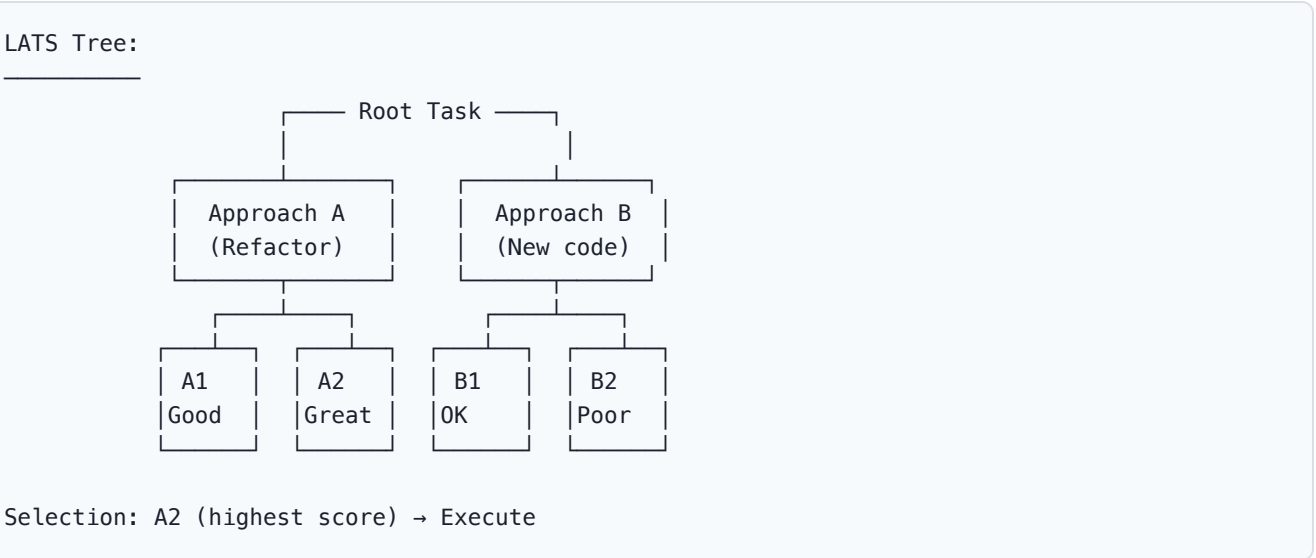
Implementación

```
def reflexion_agent(task, tools, llm, max_attempts=3):
    """Reflexion agent with self-improvement."""
    reflections = []
```

```
    for attempt in range(max_attempts):
        # Execute with reflection context
        prompt = f"""
        Task: {task}
        Previous attempts: {reflections}
        Learn from past mistakes.
        """
        result = execute_with_llm(prompt, tools, llm)

        # Evaluate
        evaluation = evaluate_result(result, task)

        if evaluation.success:
            return result

        # Reflect on failure
        reflection = llm.generate(f"""
        Task failed. Analyze why:
        Result: {result}
        Evaluation: {evaluation}
        What went wrong and how to fix it?
        """)
        reflections.append(reflection)

    return "Max attempts reached"
```

4. LATS (Language Agent Tree Search)

Combina tree search con agentes para explorar múltiples caminos.



Ventajas

- Explora múltiples soluciones

- Backtracking cuando falla
- Encuentra mejores soluciones
- Más robusto que ReAct

Limitaciones

- Más costoso (múltiples llamadas LLM)
- Más lento (explora branches)
- Complejidad de implementación

5. Tool-Use Patterns

Sequential Tool Use

```
Tool 1 → Tool 2 → Tool 3 → Result
(read)  (analyze) (write)   (verify)
```

Parallel Tool Use

```
┌──────────┐
│   Task   │
└──────────┘
      │
  ┌───┴───┐
  │       │
Tool 1  Tool 2  (execute in parallel)
  │       │
  └───┬───┘
      │
   Merge → Result
```

Conditional Tool Use

```
Task → Evaluate → Route:
  ├── Needs code? → Code tools
  ├── Needs data? → Data tools
  ├── Needs docs? → Doc tools
  └── Needs deploy? → Deploy tools
```

Selección de Arquitectura

Use Case	Recommended Architecture
Simple Q&A with tools	ReAct
Multi-step feature	Plan-and-Execute

Use Case	Recommended Architecture
Debugging complex bugs	Reflexion
Architecture decisions	LATS
CI/CD automation	Sequential Tool-use
Data analysis pipeline	Parallel Tool-use
Complex workflows	Hierarchical (multi-agent)

Referencia: Sub-agentes y Orquestación

Relación con otros conceptos

- Visión general: Agentes: Visión General
- Autonomía: Agentes Autónomos
- Orquestación: Sub-agentes y Orquestación
- Multi-agente: Sistemas Multi-agente
- MCP: Model Context Protocol
- Herramientas: Ecosistema de Herramientas

Sub Agentes Orquestacion

Sub-agentes y Orquestación

Los sub-agentes permiten delegar tareas especializadas a agentes más pequeños y enfocados, coordinados por un agente supervisor.

¿Por qué Sub-agentes?

```
Single Agent Problems:
──────────────────────

┌─────────────────────────────────────────┐
│  One agent doing everything:            │
│  • Tool overload (too many tools)       │
│  • Context window overflow              │
│  • Conflicting instructions             │
│  • Error propagation                    │
│  • Hard to debug                        │
└─────────────────────────────────────────┘

Solution: Sub-agentes
─────────────────────

┌─────────────────────────────────────────┐
│  Supervisor Agent                       │
│  ├── Coding Sub-agent                   │
│  │   └── Tools: read, write, edit       │
│  ├── Testing Sub-agent                  │
│  │   └── Tools: run tests, coverage     │
│  ├── Security Sub-agent                 │
│  │   └── Tools: scan, analyze           │
│  └── Documentation Sub-agent            │
│      └── Tools: generate docs           │
└─────────────────────────────────────────┘
```

Patterns de Orquestación

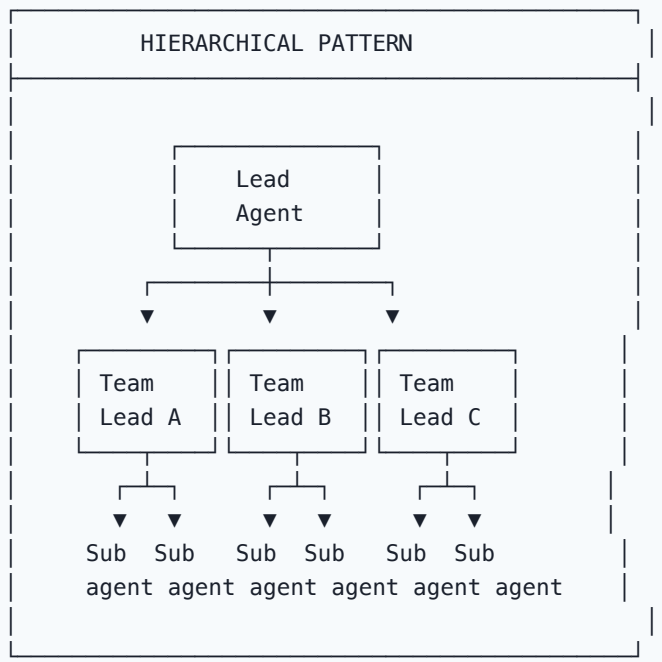
1. Supervisor Pattern

```
┌─────────────────────────────────────────────┐
│            SUPERVISOR PATTERN               │
├─────────────────────────────────────────────┤
│                                             │
│            ┌──────────────┐                 │
│            │  Supervisor  │                 │
│            │    Agent     │                 │
│            └──────┬───────┘                 │
```

Implementation

```
class SupervisorAgent:
    def __init__(self, llm):
        self.llm = llm
        self.agents = {
            "coder": CoderAgent(llm),
            "tester": TesterAgent(llm),
            "reviewer": ReviewerAgent(llm),
        }

    async def execute(self, task: str):
        # Plan
        plan = await self.plan(task)

        results = {}
        for step in plan.steps:
            agent = self.agents[step.agent_type]
            context = self.build_context(step, results)
            result = await agent.execute(step.task, context)
            results[step.id] = result

        return self.compile(results)

    async def plan(self, task: str) -> Plan:
        prompt = f"""
        Decompose this task for specialized agents:
        Task: {task}
        Available agents: {list(self.agents.keys())}

        Output: list of (agent_type, subtask, dependencies)
        """
        return await self.llm.generate(prompt)
```

2. Hierarchical Pattern



3. Sequential Pipeline

```
Input → Agent 1 → Agent 2 → Agent 3 → Output
         (Plan)    (Code)    (Test)
```

4. Parallel Fan-out

```
              ┌→ Agent A (Frontend) ─┐
Input → Split ├→ Agent B (Backend)  ─┼→ Merge → Output
              └→ Agent C (DevOps)  ─┘
```

Delegation Strategies

Task-based delegation

```
DELEGATION_RULES = {
    "code_generation": "coder_agent",
    "test_writing": "tester_agent",
    "code_review": "reviewer_agent",
    "documentation": "docs_agent",
    "security_scan": "security_agent",
```

```
    "deployment": "devops_agent",
}
```

Capability-based delegation

```
class CapabilityRouter:
    def route(self, task: Task) -> Agent:
        capabilities = self.analyze_capabilities(task)

        if capabilities.needs_code:
            return self.coder_agent
        if capabilities.needs_testing:
            return self.tester_agent
        if capabilities.needs_review:
            return self.reviewer_agent

        return self.general_agent
```

Communication Patterns

Message passing

```
# Agents communicate via structured messages
@dataclass
class AgentMessage:
    sender: str
    recipient: str
    type: str  # "task", "result", "question", "feedback"
    content: dict
    context: dict
```

Shared state

```
# All agents share a common state store
class SharedState:
    def __init__(self):
        self.codebase = {}
        self.test_results = {}
        self.review_feedback = {}
        self.metrics = {}

    def update(self, key: str, value: Any):
        self.state[key] = value

    def get(self, key: str) -> Any:
        return self.state.get(key)
```

Event-driven

```
# Agents emit and listen to events
class EventBus:
    def __init__(self):
        self.listeners = {}

    def on(self, event: str, handler: Callable):
        self.listeners.setdefault(event, []).append(handler)

    def emit(self, event: str, data: dict):
        for handler in self.listeners.get(event, []):
            handler(data)
```

Error Handling

```
Error Handling in Sub-agents:
─────────────────────────────

┌─────────────────────────────────────┐
│  Scenario: Sub-agent fails          │
│                                     │
│  Options:                           │
│  1. Retry with same agent           │
│  2. Retry with different agent      │
│  3. Skip and continue               │
│  4. Escalate to supervisor          │
│  5. Abort pipeline                  │
│                                     │
│  Strategy:                          │
│  - Transient errors → Retry (3x)    │
│  - Agent failure → Switch agent     │
│  - Critical failure → Escalate      │
│  - Budget exceeded → Abort          │
└─────────────────────────────────────┘
```

Ejemplo Completo

```
Task: "Implement user registration with email verification"

Supervisor Plan:
────────────────
1. [Coder Agent] Create User model + migration
2. [Coder Agent] Create registration endpoint
3. [Coder Agent] Create email verification service
4. [Tester Agent] Write unit tests for all new code
5. [Reviewer Agent] Security review of auth flow
6. [Docs Agent] Generate API documentation
7. [Coder Agent] Address review findings
```

```
Execution:
──────────
Step 1: Coder creates User.ts + migration ✅
Step 2: Coder creates POST /register ✅
Step 3: Coder creates EmailService.ts ✅
Step 4: Tester generates 12 test cases ✅
Step 5: Reviewer finds 2 issues:
  - Missing rate limiting on registration
  - No email format validation
Step 6: Docs generates OpenAPI spec ✅
Step 7: Coder fixes both issues ✅

Result: Feature complete with tests, review, and docs
```

Best Practices

1. **Clear responsibilities**: Cada sub-agente tiene un scope definido
2. **Minimal tool access**: Solo las tools que necesita
3. **Structured communication**: Messages con schema definido
4. **Checkpoint & rollback**: Ability to undo sub-agent changes
5. **Budget limits**: Max tokens/time per sub-agent
6. **Monitoring**: Log de todas las interacciones

Relación con otros conceptos

- Visión general: Agentes: Visión General
- Arquitecturas: Arquitecturas de Agentes
- Multi-agente: Sistemas Multi-agente
- MCP: Model Context Protocol
- Herramientas: Ecosistema de Herramientas

Multi Agent Systems

Sistemas Multi-agente

Los sistemas multi-agente coordinan múltiples agentes independientes para resolver tareas complejas que un solo agente no podría manejar.

Frameworks Principales

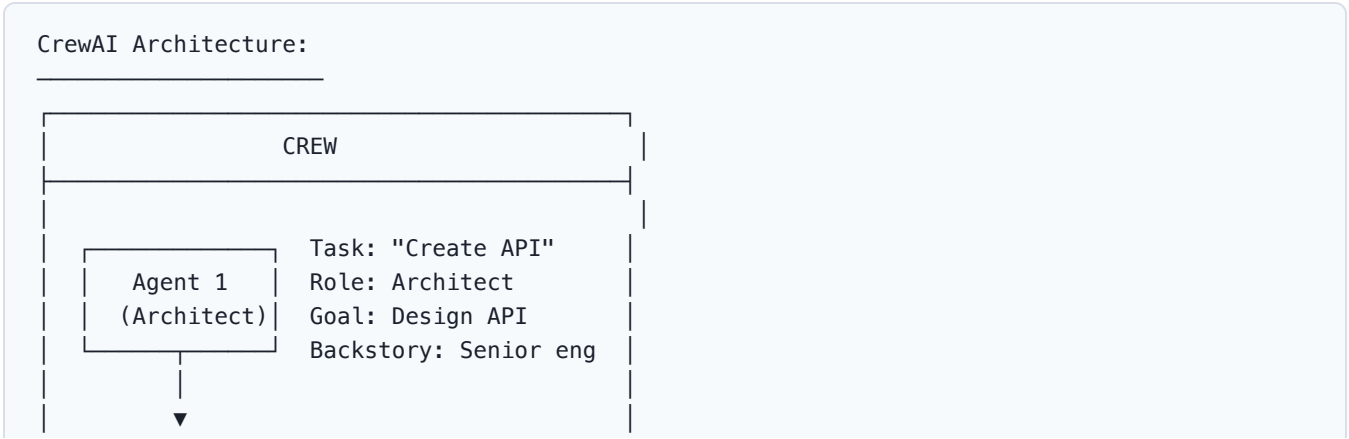
Comparison Matrix

Feature	CrewAI	AutoGen	LangGraph	Swarm
Paradigm	Role-based	Conversation	Graph-based	Handoff
Complexity	Low-Medium	Medium	High	Low
Learning curve	Easy	Medium	Steep	Easy
Flexibility	Medium	High	Very High	Low
Production ready	Yes	Yes	Yes	No (experimental)
Memory	Built-in	Built-in	Custom	Limited
Tool support	Excellent	Good	Excellent	Basic
LangSmith integration	Yes	No	Yes	No
Best for	Teams, roles	Research, debate	Complex workflows	Simple delegation

1. CrewAI

Concept

CrewAI modela agentes como "miembros de un equipo" con roles específicos.



Ejemplo de código

```
from crewai import Agent, Task, Crew

# Define agents
architect = Agent(
    role="Software Architect",
    goal="Design scalable API architecture",
    backstory="Senior architect with 15 years experience",
    tools=[read_file_tool, search_code_tool],
    llm="gpt-4o"
)

developer = Agent(
    role="Senior Developer",
    goal="Implement clean, testable code",
    backstory="Full-stack developer expert in TypeScript",
    tools=[read_file_tool, write_file_tool, run_tests_tool],
    llm="claude-3-opus"
)

reviewer = Agent(
    role="Code Reviewer",
    goal="Ensure code quality and security",
    backstory="Security-focused QA engineer",
    tools=[read_file_tool, search_code_tool],
    llm="gpt-4o"
)

# Define tasks
design_task = Task(
    description="Design REST API for user management",
```

```
    expected_output="API spec with endpoints, schemas",
    agent=architect
)

implement_task = Task(
    description="Implement the API based on the design",
    expected_output="Working API with tests",
    agent=developer,
    context=[design_task]
)

review_task = Task(
    description="Review implementation for issues",
    expected_output="List of issues and suggestions",
    agent=reviewer,
    context=[design_task, implement_task]
)

# Create and run crew
crew = Crew(
    agents=[architect, developer, reviewer],
    tasks=[design_task, implement_task, review_task],
    verbose=True
)

result = crew.kickoff()
```

2. AutoGen (Microsoft)

Concept

AutoGen usa conversaciones entre agentes para resolver tareas.

```
AutoGen Conversation:
─────────────────────

┌─────────────────────────────────────────┐
│  User Proxy ←→ Assistant Agent          │
│       ↕              ↕                  │
│  Code Executor ←→ Critic Agent          │
│                                         │
│  Flow:                                  │
│  1. User defines task                   │
│  2. Assistant proposes solution         │
│  3. Code Executor runs code             │
│  4. Critic evaluates result             │
│  5. Iterate until satisfactory          │
└─────────────────────────────────────────┘
```

Ejemplo

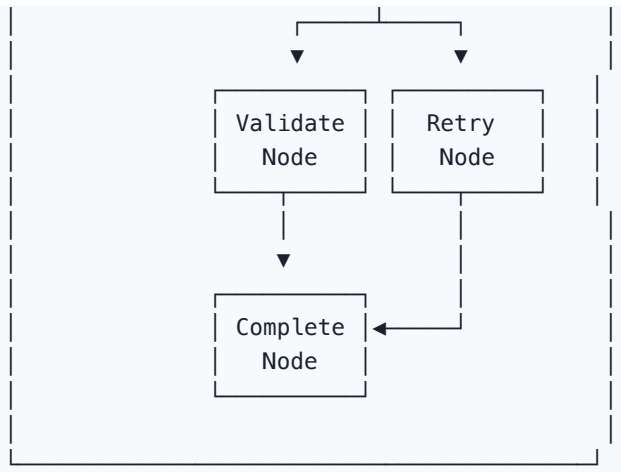
```
from autogen import AssistantAgent, UserProxyAgent, GroupChat

# Create agents
assistant = AssistantAgent(
    name="coder",
    system_message="You are an expert Python developer.",
    llm_config={"model": "gpt-4o"}
)

critic = AssistantAgent(
    name="critic",
    system_message="You review code and suggest improvements.",
    llm_config={"model": "gpt-4o"}
)

user_proxy = UserProxyAgent(
    name="user",
    human_input_mode="NEVER",
    code_execution_config={"work_dir": "coding"}
)

# Group chat
group_chat = GroupChat(
    agents=[user_proxy, assistant, critic],
    messages=[],
    max_round=10
)

# Start conversation
user_proxy.initiate_chat(
    group_chat,
    message="Create a REST API with FastAPI"
)
```

3. LangGraph

Concept

LangGraph modela agentes como nodos en un grafo con edges condicionales.

```
LangGraph Graph:
─────────────────

┌─────────────────────────────────────────┐
│                                         │
│   ┌──────────┐     ┌──────────────┐     │
│   │  Plan    │────►│   Execute    │     │
│   │  Node    │     │    Node      │     │
│   └──────────┘     └──────┬───────┘     │
│                           │             │
```



Ejemplo

```
from langgraph.graph import StateGraph, END
from typing import TypedDict, Annotated

class AgentState(TypedDict):
    task: str
    plan: list
    results: list
    attempt: int

def plan_node(state: AgentState):
    # Create plan
    plan = create_plan(state["task"])
    return {"plan": plan}

def execute_node(state: AgentState):
    # Execute current step
    result = execute_step(state["plan"][0])
    return {"results": state["results"] + [result]}

def validate_node(state: AgentState):
    # Validate results
    if all_valid(state["results"]):
        return "complete"
    return "retry"

def retry_node(state: AgentState):
    # Retry failed steps
    return {"attempt": state["attempt"] + 1}

# Build graph
graph = StateGraph(AgentState)
graph.add_node("plan", plan_node)
graph.add_node("execute", execute_node)
```

```
graph.add_node("validate", validate_node)
graph.add_node("retry", retry_node)

graph.add_edge("plan", "execute")
graph.add_edge("execute", "validate")
graph.add_conditional_edges("validate", {
    "complete": END,
    "retry": "retry"
})
graph.add_edge("retry", "execute")

app = graph.compile()
result = app.invoke({"task": "Add auth", "plan": [], "results": [], "attempt": 0})
```

Multi-agent Patterns

Sequential (Pipeline)

```
Agent A → Agent B → Agent C → Result
(Plan)   (Code)   (Review)
```

Parallel (Fan-out/Fan-in)

```
          ┌→ Agent A ─┐
Split ──  ├→ Agent B ─┼── Merge → Result
          └→ Agent C ─┘
```

Hierarchical (Tree)

```
          ┌→ Team A ──→ Agent A1
Leader ──┼→ Team B ──→ Agent B1
          └→ Team C ──→ Agent C1
```

Debate/Consensus

```
Agent A ←→ Agent B ←→ Agent C
   ↓           ↓           ↓
   └──────── Consensus ──────┘
```

Casos de Uso en Software Factory

Use Case	Pattern	Framework
Feature implementation	Sequential	CrewAI

Use Case	Pattern	Framework
Code review + fix	Debate	AutoGen
Complex refactoring	Hierarchical	LangGraph
Multi-service deployment	Parallel	LangGraph
Architecture decision	Debate	AutoGen
Bug investigation	Sequential	CrewAI
Documentation + testing	Parallel	CrewAI

Best Practices

1. **Start simple**: Un solo agente primero, escalar después
2. **Clear roles**: Cada agente tiene un propósito claro
3. **Minimize handoffs**: Menos comunicaciones = menos errores
4. **Shared context**: Estado compartido entre agentes
5. **Error isolation**: Un agente fallando no debe parar todo
6. **Cost control**: Monitor token usage por agente
7. **Observability**: Log de todas las interacciones

Relación con otros conceptos

- Orquestación: Sub-agentes y Orquestación
- Arquitecturas: Arquitecturas de Agentes
- Visión general: Agentes: Visión General
- MCP: Model Context Protocol
- Herramientas: Ecosistema de Herramientas

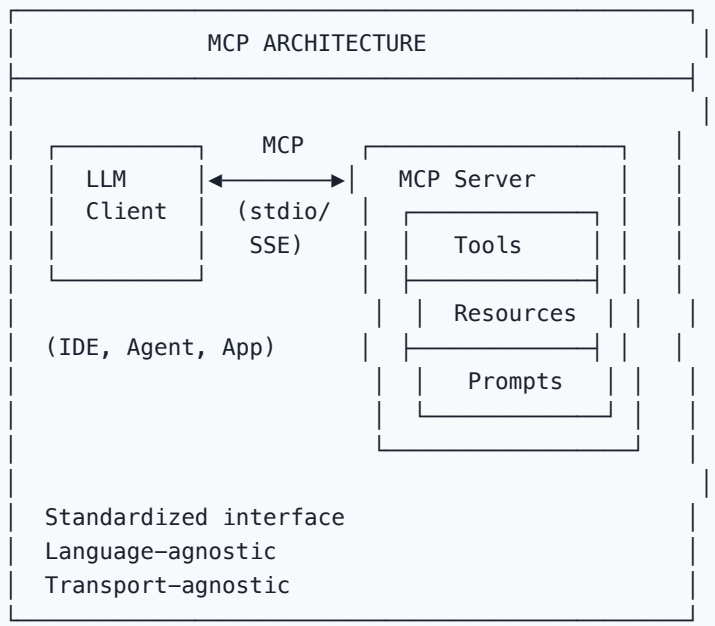
Mcp Protocol

Model Context Protocol (MCP)

MCP es un protocolo abierto que estandariza cómo los LLMs se conectan a fuentes de datos y herramientas externas. Es el "USB-C" de los agentes de IA.

¿Qué es MCP?

> MCP (Model Context Protocol) es un protocolo abierto que define una interfaz estándar para conectar LLMs con contextos externos: tools, resources y prompts.



Componentes MCP

Client-Server Model

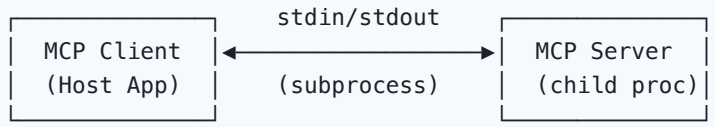
```
Host Application (IDE, Agent)
    │
    ├── MCP Client 1 ──→ MCP Server A (filesystem)
    │
    ├── MCP Client 2 ──→ MCP Server B (database)
    │
    └── MCP Client 3 ──→ MCP Server C (API)
```

Three Core Primitives

Primitive	Description	Analogy
Tools	Functions the LLM can call	POST endpoints
Resources	Data the LLM can read	GET endpoints
Prompts	Pre-defined interaction templates	API schemas

Transport Layers

1. stdio (Standard I/O)



Use case: Local tools, CLI-based servers

Ejemplo de server stdio:

```
import { Server } from "@modelcontextprotocol/sdk/server";
import { StdioServerTransport } from "@modelcontextprotocol/sdk/server/stdio";

const server = new Server(
  { name: "filesystem", version: "1.0.0" },
  { capabilities: { tools: {}, resources: {} } }
);

// Register tools
server.setRequestHandler("tools/list", async () => ({
  tools: [
    {
      name: "read_file",
      description: "Read contents of a file",
      inputSchema: {
        type: "object",
        properties: {
          path: { type: "string", description: "File path" }
        },
        required: ["path"]
      }
    }
  ]
}));

// Start with stdio transport
```

```
const transport = new StdioServerTransport();
await server.connect(transport);
```

2. SSE (Server-Sent Events)


```
Use case: Remote servers, web-based clients
```

3. Streamable HTTP (nuevo en 2025)

```
POST /mcp → Full-duplex streaming
Supports: JSON-RPC over HTTP with streaming responses
```

Lifecycle

```
MCP Connection Lifecycle:
─────────────────────────

1. INITIALIZATION
   ├── Client sends: initialize
   ├── Server responds: capabilities
   └── Client sends: initialized

2. OPERATION
   ├── Client calls tools
   ├── Client reads resources
   └── Client uses prompts

3. SHUTDOWN
   ├── Client sends: close
   └── Server cleans up
```

Handshake example

```
// Client → Server: initialize
{
  "jsonrpc": "2.0",
  "method": "initialize",
  "params": {
    "protocolVersion": "2025-03-26",
    "capabilities": {
      "tools": {},
      "resources": {}
```

```
    },
    "clientInfo": {
      "name": "my-ide",
      "version": "1.0.0"
    }
  }
}

// Server → Client: initialize response
{
  "jsonrpc": "2.0",
  "result": {
    "protocolVersion": "2025-03-26",
    "capabilities": {
      "tools": { "listChanged": true },
      "resources": { "subscribe": true }
    },
    "serverInfo": {
      "name": "filesystem-server",
      "version": "1.0.0"
    }
  }
}
```

Capabilities

Capability	Description	Server declares
`tools`	Can call tools	
`resources`	Can read resources	
`prompts`	Has prompt templates	
`sampling`	Can request LLM sampling	
`logging`	Can send log messages	

MCP vs Alternativas

Aspect	MCP	Function Calling	LangChain Tools	Custom API
Standard	Open protocol	Vendor-specific	Framework-specific	Custom
Interoperability	High	Low	Medium	Low
Discovery	Built-in	Manual	Manual	Manual
Streaming	Yes	Varies	Varies	Custom
Resources	Yes	No	No	Custom

Aspect	MCP	Function Calling	LangChain Tools	Custom API
Prompts	Yes	No	No	No
Ecosystem	Growing	Large	Large	None

MCP Ecosystem (2026)

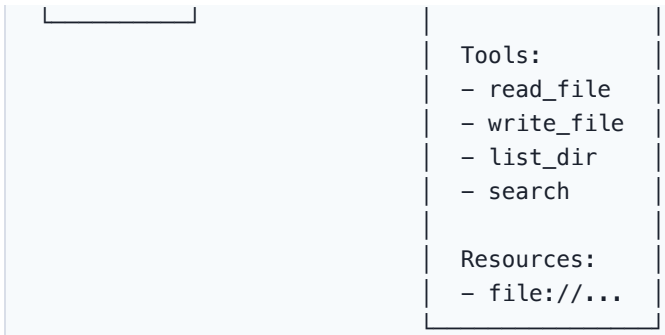
Servers disponibles

Server	Provider	Purpose
filesystem	MCP	File read/write
github	MCP	GitHub API
postgres	MCP	Database queries
slack	MCP	Slack messaging
puppeteer	MCP	Browser automation
memory	MCP	Persistent memory
brave-search	MCP	Web search
google-maps	MCP	Location services

Clients que soportan MCP

Client	Type
Claude Desktop	Desktop app
Cursor	IDE
Continue.dev	IDE
Zed	IDE
Cline	VS Code extension
Opencode	CLI tool

Ejemplo: Filesystem Server



Configuración en Claude Desktop

```
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-filesystem",
        "/Users/me/projects"
      ]
    }
  }
}
```

Best Practices

1. **Start with existing servers**: No reinvent the wheel
2. **Minimal permissions**: Solo los paths/herramientas necesarios
3. **Error handling**: Servers deben manejar errores gracefully
4. **Logging**: Habilitar para debugging
5. **Version control**: Mantener protocol version actualizado
6. **Security**: Validar inputs, sanitizar outputs

Relación con otros conceptos

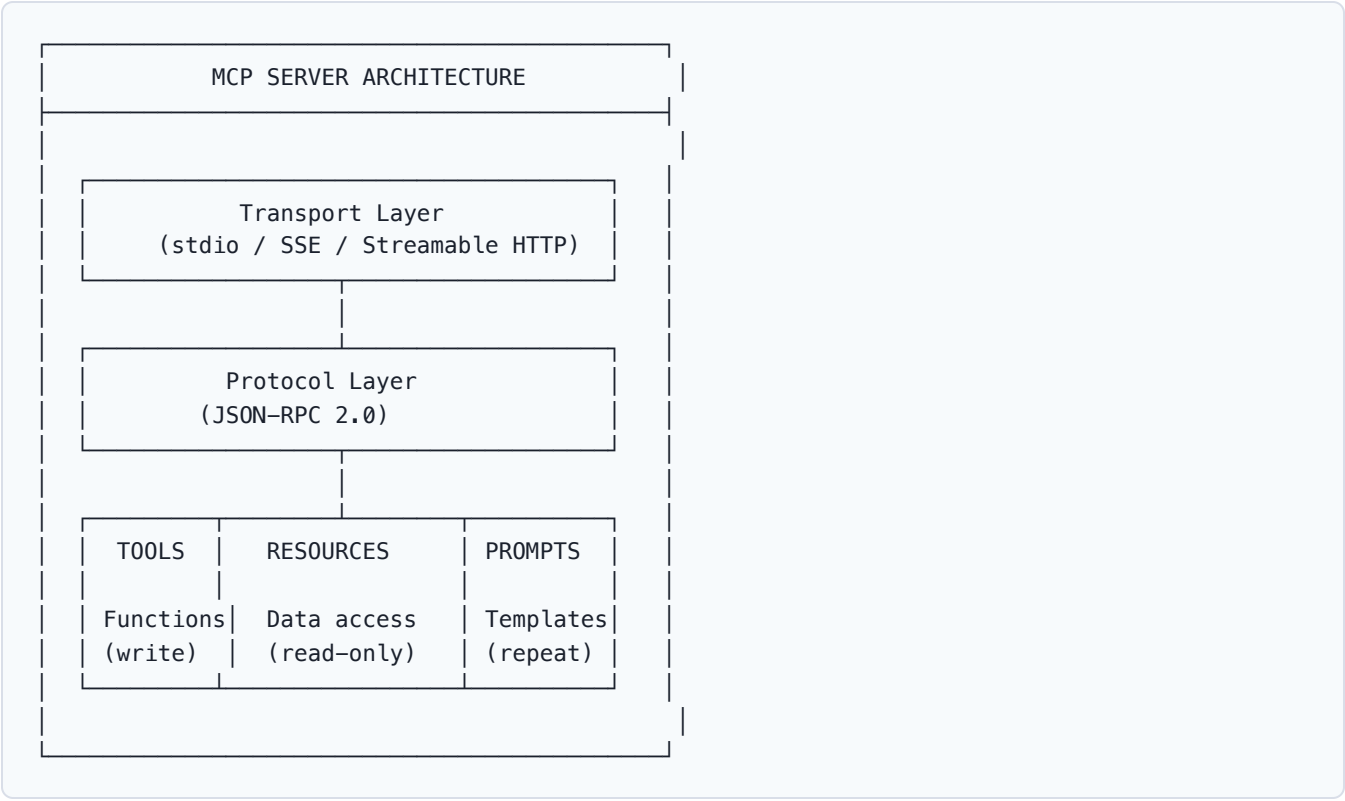
- Servers/Tools: MCP Servers, Tools y Resources
- Agentes: Agentes de IA
- Herramientas: Ecosistema de Herramientas
- Automatización: Scripts y Automatización

Mcp Servers Tools

MCP Servers, Tools y Resources

MCP Servers exponen Tools, Resources y Prompts que los LLMs pueden usar. Este artículo detalla cómo diseñarlos y usarlos.

MCP Server Architecture



Tools (Herramientas)

Definición

Tools son funciones que el LLM puede invocar. Toman input estructurado y retornan resultados.

```
// Tool definition
{
  name: "create_issue",
  description: "Create a GitHub issue",
  inputSchema: {
    type: "object",
    properties: {
      title: {
```

```
        type: "string",
        description: "Issue title"
      },
      body: {
        type: "string",
        description: "Issue body (markdown)"
      },
      labels: {
        type: "array",
        items: { type: "string" },
        description: "Labels to add"
      }
    },
    required: ["title"]
  }
}
```

Tool Implementation

```
// Server-side handler
server.setRequestHandler("tools/call", async (request) => {
  const { name, arguments: args } = request.params;

  switch (name) {
    case "create_issue":
      const issue = await github.createIssue({
        owner: "myorg",
        repo: "myrepo",
        title: args.title,
        body: args.body,
        labels: args.labels || []
      });

      return {
        content: [{
          type: "text",
          text: `Created issue #${issue.number}: ${issue.title}`
        }]
      };

    case "search_code":
      const results = await searchCode(args.pattern, args.path);
      return {
        content: [{
          type: "text",
          text: JSON.stringify(results, null, 2)
        }]
      };

    default:
```

```
      throw new Error(`Unknown tool: ${name}`);
  }
});
```

Tool Patterns

```
Tool Categories:
─────────────────
1. Read-only tools (safe)
   ├── read_file
   ├── search_code
   ├── list_directory
   └── get_weather

2. Write tools (need validation)
   ├── write_file
   ├── create_issue
   ├── send_email
   └── deploy

3. Execution tools (high risk)
   ├── run_command
   ├── execute_sql
   ├── run_tests
   └── build_project
```

Resources (Recursos)

Definición

Resources exponen datos que el LLM puede leer. Son READ-ONLY por diseño.

```
// Resource definition
{
  uri: "file:///path/to/project/src",
  name: "Source Directory",
  description: "Project source code directory",
  mimeType: "text/plain"
}

// Resource template
{
  uriTemplate: "file:///path/to/project/{filepath}",
  name: "Project File",
  description: "Read any file in the project",
  mimeType: "text/plain"
}
```

Resource Implementation

```
// List available resources
server.setRequestHandler("resources/list", async () => ({
  resources: [
    {
      uri: "config://app/settings",
      name: "App Settings",
      description: "Application configuration",
      mimeType: "application/json"
    },
    {
      uri: "db://schema/tables",
      name: "Database Schema",
      description: "Database table definitions",
      mimeType: "application/json"
    }
  ]
}));

// Read resource
server.setRequestHandler("resources/read", async (request) => {
  const { uri } = request.params;

  if (uri === "config://app/settings") {
    const config = await readConfig();
    return {
      contents: [{
        uri,
        mimeType: "application/json",
        text: JSON.stringify(config, null, 2)
      }]
    };
  }
});
```

Resource vs Tool

Aspect	Resource	Tool
Operation	Read-only	Read/Write
Side effects	None	Possible
Caching	Aggressive	Careful
Use case	Data retrieval	Actions
Safety	High	Requires validation

Prompts

Definición

Prompts son templates de interacción pre-definidos.

```
// Prompt definition
{
  name: "code_review",
  description: "Review code for issues",
  arguments: [
    {
      name: "language",
      description: "Programming language",
      required: true
    },
    {
      name: "focus",
      description: "Review focus area",
      required: false
    }
  ]
}
```

Prompt Implementation

```
server.setRequestHandler("prompts/list", async () => ({
  prompts: [
    {
      name: "code_review",
      description: "Review code for quality and security",
      arguments: [
        { name: "language", description: "Language", required: true },
        { name: "focus", description: "Focus area", required: false }
      ]
    },
    {
      name: "debug_error",
      description: "Help debug an error",
      arguments: [
        { name: "error_message", description: "Error message", required: true },
        { name: "stack_trace", description: "Stack trace", required: false }
      ]
    }
  ]
}));

server.setRequestHandler("prompts/get", async (request) => {
  const { name, arguments: args } = request.params;
```

```
  if (name === "code_review") {
    return {
      messages: [
        {
          role: "user",
          content: {
            type: "text",
            text: `Review this ${args.language} code for:
- Security vulnerabilities
- Performance issues
- Code quality
${args.focus ? `Focus on: ${args.focus}` : ""}`
          }
        }
      ]
    };
  }
});
```

Sampling

Sampling permite al servidor pedir al LLM que genere contenido.

```
// Server requests LLM generation
const result = await server.requestSampling({
  messages: [{
    role: "user",
    content: {
      type: "text",
      text: "Generate a test case for this function: " + functionCode
    }
  }],
  modelPreferences: {
    hints: [{ name: "claude-3-opus" }]
  },
  maxTokens: 1024
});
```

Ejemplo: Database Server Completo

```
import { Server } from "@modelcontextprotocol/sdk/server";
import { StdioServerTransport } from "@modelcontextprotocol/sdk/server/stdio";
import pg from "pg";

const pool = new pg.Pool({ connectionString: process.env.DATABASE_URL });

const server = new Server(
  { name: "postgres", version: "1.0.0" },
```

```
  { capabilities: { tools: {}, resources: {} } }
);

// Tools
server.setRequestHandler("tools/list", async () => ({
  tools: [
    {
      name: "query",
      description: "Execute a read-only SQL query",
      inputSchema: {
        type: "object",
        properties: {
          sql: { type: "string", description: "SQL query" }
        },
        required: ["sql"]
      }
    },
    {
      name: "list_tables",
      description: "List all tables in the database",
      inputSchema: { type: "object", properties: {} }
    }
  ]
}));

server.setRequestHandler("tools/call", async (request) => {
  const { name, arguments: args } = request.params;

  if (name === "query") {
    // Security: only allow SELECT
    if (!args.sql.trim().toUpperCase().startsWith("SELECT")) {
      return {
        content: [{ type: "text", text: "Error: Only SELECT queries allowed" }],
        isError: true
      };
    }

    const result = await pool.query(args.sql);
    return {
      content: [{
        type: "text",
        text: JSON.stringify(result.rows, null, 2)
      }]
    };
  }

  if (name === "list_tables") {
    const result = await pool.query(`
      SELECT table_name FROM information_schema.tables
      WHERE table_schema = 'public'
    `);
```

```
    return {
      content: [{
        type: "text",
        text: result.rows.map(r => r.table_name).join("\n")
      }]
    };
  }
});

// Resources
server.setRequestHandler("resources/list", async () => ({
  resources: [{
    uri: "db://schema",
    name: "Database Schema",
    description: "Full database schema",
    mimeType: "application/json"
  }]
}));

const transport = new StdioServerTransport();
await server.connect(transport);
```

Best Practices

1. **Minimal exposure**: Solo las tools necesarias
2. **Input validation**: Validar todos los inputs
3. **Error messages**: Claros y accionables
4. **Rate limiting**: Prevenir abuse
5. **Logging**: Para debugging y auditoría
6. **Documentation**: Buenas descriptions para el LLM

Relación con otros conceptos

- Protocolo: Model Context Protocol
- Agentes: Agentes de IA
- Automatización: Scripts y Automatización
- Herramientas: Ecosistema de Herramientas

Ai Tools Ecosystem

Ecosistema de Herramientas AI

El ecosistema de herramientas AI para desarrollo es amplio y en constante evolución. Este artículo compara los frameworks principales.

Panorama

```
┌─────────────────────────────────────────────────────┐
│           AI TOOLS ECOSYSTEM (2026)                 │
├──────────────┬──────────────┬───────────────────────┤
│  ORCHESTRATE │  RETRIEVE    │  BUILD                │
│              │              │                       │
│  LangChain   │ LlamaIndex   │ Semantic Kernel       │
│  LangGraph   │ Haystack     │ OpenAI SDK            │
│  CrewAI      │ Vectara      │ Anthropic SDK         │
│  AutoGen     │ Weaviate     │ Vercel AI SDK         │
│  Mastra      │ Pinecone     │ Google AI Edge        │
└──────────────┴──────────────┴───────────────────────┘
```

1. LangChain

¿Qué es?

Framework para construir aplicaciones powered por LLMs con components modulares.

```
LangChain Architecture:
─────────────────────────

┌──────────────────────────────────────┐
│           LangChain Ecosystem        │
├──────────────────────────────────────┤
│  langchain-core    → Base components│
│  langchain         → Chains, agents  │
│  langchain-community → Integrations  │
│  langgraph         → Agent workflows│
│  langsmith         → Observability   │
└──────────────────────────────────────┘
```

Ejemplo

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnablePassthrough
```

```
# Simple chain
llm = ChatOpenAI(model="gpt-4o")
prompt = ChatPromptTemplate.from_template(
    "Explain {concept} to a {audience}"
)
parser = StrOutputParser()

chain = prompt | llm | parser
result = chain.invoke({"concept": "MCP", "audience": "junior developer"})
```

Ventajas/Limitaciones

Ventajas	Limitaciones
Ecosistema enorme	Abstracciones pesadas
Muchas integraciones	Learning curve alta
LangSmith para debugging	Overhead de dependencias
Community activo	API cambia frecuentemente

2. LlamaIndex

¿Qué es?

Framework especializado en RAG (Retrieval-Augmented Generation) y data indexing.

```
LlamaIndex Flow:
─────────────────
Documents → Index → Query → Context → LLM → Response
              │
              ├── Vector Store
              ├── Keyword Index
              └── Knowledge Graph
```

Ejemplo

```
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader
from llama_index.llms.openai import OpenAI

# Load documents
documents = SimpleDirectoryReader("./docs").load_data()

# Create index
index = VectorStoreIndex.from_documents(documents)

# Query
```

```
query_engine = index.as_query_engine(llm=OpenAI(model="gpt-4o"))
response = query_engine.query("What is our API authentication method?")
print(response)
```

Best for

- Document Q&A systems
- Knowledge bases
- Search with context
- Enterprise data integration

3. Semantic Kernel (Microsoft)

¿Qué es?

Framework de Microsoft para integrar LLMs en aplicaciones .NET/Python/Java.

```
Semantic Kernel:
─────────────────

┌─────────────────────────────────────┐
│  Kernel                             │
│  ├── Plugins (Functions)            │
│  │   ├── Native functions (code)    │
│  │   └── Semantic functions (prompt)│
│  ├── Memory                         │
│  ├── Planner                        │
│  └── AI Services                    │
│      ├── Chat completion            │
│      ├── Text generation            │
│      └── Embeddings                 │
└─────────────────────────────────────┘
```

Ejemplo

```
import semantic_kernel as sk
from semantic_kernel.connectors.ai.open_ai import OpenAIChatCompletion

kernel = sk.Kernel()
kernel.add_service(OpenAIChatCompletion("gpt-4o", api_key))

# Register a plugin
@sk.plugin
class MathPlugin:
    @sk.function(description="Add two numbers")
    def add(self, a: float, b: float) -> float:
        return a + b

kernel.add_plugin(MathPlugin(), "math")
```

```
# Use planner
planner = sk.FunctionCallingStepwisePlanner()
result = await planner.invoke(kernel, "What is 123 + 456?")
```

4. Haystack (deepset)

Framework para NLP pipelines

```
from haystack import Pipeline
from haystack.components.generators import OpenAIGenerator
from haystack.components.retrievers import InMemoryBM25Retriever
from haystack.components.writers import DocumentWriter

pipeline = Pipeline()
pipeline.add_component("retriever", InMemoryBM25Retriever())
pipeline.add_component("llm", OpenAIGenerator(model="gpt-4o"))
pipeline.connect("retriever", "llm")
```

5. Vercel AI SDK

Para aplicaciones web

```
import { generateText, streamText } from 'ai';
import { openai } from '@ai-sdk/openai';

// Simple generation
const { text } = await generateText({
  model: openai('gpt-4o'),
  prompt: 'Explain MCP protocol'
});

// Streaming
const stream = streamText({
  model: openai('gpt-4o'),
  prompt: 'Write a REST API'
});

for await (const chunk of stream.textStream) {
  process.stdout.write(chunk);
}
```

Comparison Matrix

Feature	LangChain	LlamaIndex	Semantic Kernel	Haystack	Vercel AI
Primary use	Orchestration	RAG	Enterprise apps	NLP pipelines	Web apps
Languages	Python/JS	Python	.NET/Py/Java	Python	TypeScript
Learning curve	High	Medium	Medium	Medium	Low
RAG support	Good	Excellent	Good	Good	Basic
Agent support	Excellent (LangGraph)	Basic	Good	Basic	Basic
MCP support	Via LangChain	No	No	No	No
Production ready	Yes	Yes	Yes	Yes	Yes
Cloud offering	LangSmith	LlamaCloud	Azure AI	deepset Cloud	Vercel
Community size	Very large	Large	Large	Medium	Growing
Best for	Complex agents	Document search	Microsoft stack	Enterprise NLP	Next.js apps

Decision Framework

```
¿Qué necesitas construir?
─────────────────────────

¿Sistema de Q&A sobre documentos?
  → LlamaIndex (RAG especializado)

¿Agente complejo con múltiples tools?
  → LangChain + LangGraph

¿Aplicación .NET/Enterprise Microsoft?
  → Semantic Kernel

¿Pipeline de NLP production-grade?
  → Haystack

¿App web con AI streaming?
  → Vercel AI SDK

¿MCP integration?
  → LangChain o MCP SDK directo

¿Máxima flexibilidad?
  → Build with raw SDK (OpenAI/Anthropic)
```

MCP Integration

Referencia: Model Context Protocol

Best Practices

1. **Evaluate antes de adoptar**: No usar el más popular, sino el más adecuado
2. **Start simple**: Raw SDK primero, frameworks si es necesario
3. **Observability**: Usar LangSmith o similar desde el día 1
4. **Testing**: Tests para chains, agents y tools
5. **Version pinning**: Lock versions en producción

Relación con otros conceptos

- MCP: Model Context Protocol
- Agentes: Agentes de IA
- Herramientas: AI Coding Assistants
- Scripts: Scripts y Automatización

Ai Scripts Automation

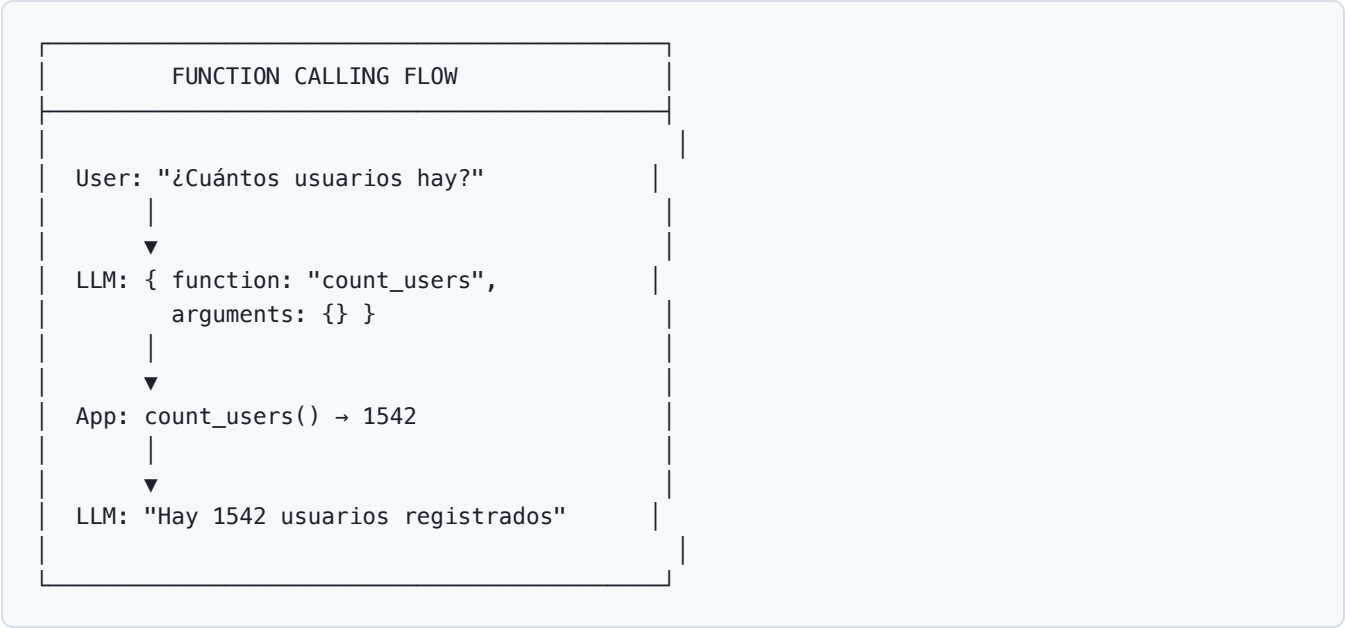
Scripts y Automatización con IA

La IA permite crear scripts inteligentes que pueden tomar decisiones, adaptarse a contexto y ejecutar tareas complejas.

Function Calling

¿Qué es?

Function calling permite al LLM invocar funciones definidas por el developer.



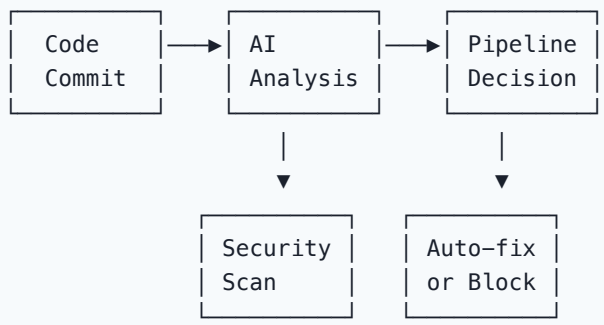
OpenAI Function Calling

```
import openai

tools = [
    {
        "type": "function",
        "function": {
            "name": "get_weather",
            "description": "Get weather for a city",
            "parameters": {
                "type": "object",
                "properties": {
                    "city": {
                        "type": "string",
                        "description": "City name"
```

```
                    }
                },
                "required": ["city"]
            }
        }
    }
]

response = openai.chat.completions.create(
    model="gpt-4o",
    messages=[{"role": "user", "content": "Weather in Madrid?"}],
    tools=tools,
    tool_choice="auto"
)

# Handle function call
if response.choices[0].message.tool_calls:
    tool_call = response.choices[0].message.tool_calls[0]
    if tool_call.function.name == "get_weather":
        import json
        args = json.loads(tool_call.function.arguments)
        result = get_weather(args["city"])
        print(result)
```

Automation Patterns

1. CI/CD Pipeline Enhancement




```
# .github/workflows/ai-enhanced.yml
name: AI-Enhanced CI
on: [push, pull_request]

jobs:
  ai-analysis:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
```

```
      - name: AI Code Review
        uses: coderabbitai/ai-pr-reviewer@latest
        env:
          GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}

      - name: AI Security Scan
        run: |
          # Custom AI security analysis
          python scripts/ai_security_scan.py

      - name: AI Test Suggestions
        if: github.event_name == 'pull_request'
        run: |
          # Suggest tests for changed files
          python scripts/ai_suggest_tests.py --diff HEAD~1
```

2. Automated Documentation

```
# Script: auto_document.py
import openai
import os

def document_function(file_path: str, function_name: str):
    """Auto-generate documentation for a function."""
    with open(file_path) as f:
        code = f.read()

    response = openai.chat.completions.create(
        model="gpt-4o",
        messages=[{
            "role": "user",
            "content": f"""Generate comprehensive JSDoc for:
{code}

Include: description, params, returns, examples, throws"""
        }]
    )

    return response.choices[0].message.content

# Batch documentation
for root, dirs, files in os.walk("src"):
    for file in files:
        if file.endswith(".ts"):
            path = os.path.join(root, file)
            docs = document_function(path, "all")
            # Write to docs/ directory
```

3. Intelligent Monitoring

```
# Script: ai_monitor.py
def analyze_log(log_entry: dict) -> dict:
    """AI-powered log analysis."""
    response = openai.chat.completions.create(
        model="gpt-4o",
        messages=[{
            "role": "system",
            "content": """Analyze log entries. Return JSON:
            {severity, category, suggested_action, is_known_issue}"""
        }, {
            "role": "user",
            "content": f"Analyze: {log_entry}"
        }],
        response_format={"type": "json_object"}
    )

    return json.loads(response.choices[0].message.content)
```

4. Migration Scripts

```
# Script: ai_migrate.py
def generate_migration(source_code: str, target: str) -> str:
    """AI-powered code migration."""
    response = openai.chat.completions.create(
        model="gpt-4o",
        messages=[{
            "role": "system",
            "content": f"Migrate code to {target}. Preserve all functionality."
        }, {
            "role": "user",
            "content": source_code
        }]
    )
    return response.choices[0].message.content

# Batch migrate
for file in glob("legacy/**/*.py"):
    source = read_file(file)
    migrated = generate_migration(source, "FastAPI")
    write_file(f"modern/{file.name}", migrated)
```

MCP-Based Automation

```
┌─────────────────────────────────────────────┐
│      MCP AUTOMATION ARCHITECTURE            │
├─────────────────────────────────────────────┤
```

```
┌                                                  ┐
│  ┌──────────┐     ┌──────────────────┐           │
│  │  Agent   │────►│  MCP Server      │           │
│  │  (LLM)   │◄────│  - GitHub API    │           │
│  └──────────┘     │  - Database      │           │
│                   │  - File System   │           │
│                   │  - Deploy Tools  │           │
│                   └──────────────────┘           │
│                                                  │
│  Example workflow:                               │
│  "Deploy the latest changes to staging"          │
│  → Agent reads GitHub for latest commit          │
│  → Agent runs tests via MCP                      │
│  → Agent deploys via MCP deploy tool             │
│  → Agent notifies via Slack MCP                  │
└──────────────────────────────────────────────────┘
```

CLI Automation

Custom CLI tool with AI

```python
# cli.py - Custom AI-powered CLI
import click
import openai

@click.group()
def cli():
    pass

@cli.command()
@click.argument('issue')
def investigate(issue):
    """AI-powered issue investigation."""
    click.echo(f"Investigating: {issue}")

    # Search codebase
    results = search_code(issue)

    # AI analysis
    analysis = openai.chat.completions.create(
        model="gpt-4o",
        messages=[{
            "role": "user",
            "content": f"Investigate this issue: {issue}\n\nCode context:\n{results}"
        }]
    )

    click.echo(analysis.choices[0].message.content)
```

```
@cli.command()
@click.argument('file_path')
def explain(file_path):
    """Explain code in a file."""
    code = read_file(file_path)
    response = openai.chat.completions.create(
        model="gpt-4o",
        messages=[{
            "role": "user",
            "content": f"Explain this code:\n\n{code}"
        }]
    )
    click.echo(response.choices[0].message.content)
```

Best Practices

1. **Error handling**: AI calls can fail, always handle gracefully
2. **Cost control**: Monitor token usage in scripts
3. **Rate limiting**: Respect API rate limits
4. **Caching**: Cache AI responses when appropriate
5. **Logging**: Log all AI interactions for debugging
6. **Timeouts**: Set appropriate timeouts for AI calls
7. **Fallbacks**: Have fallback when AI is unavailable

Relación con otros conceptos

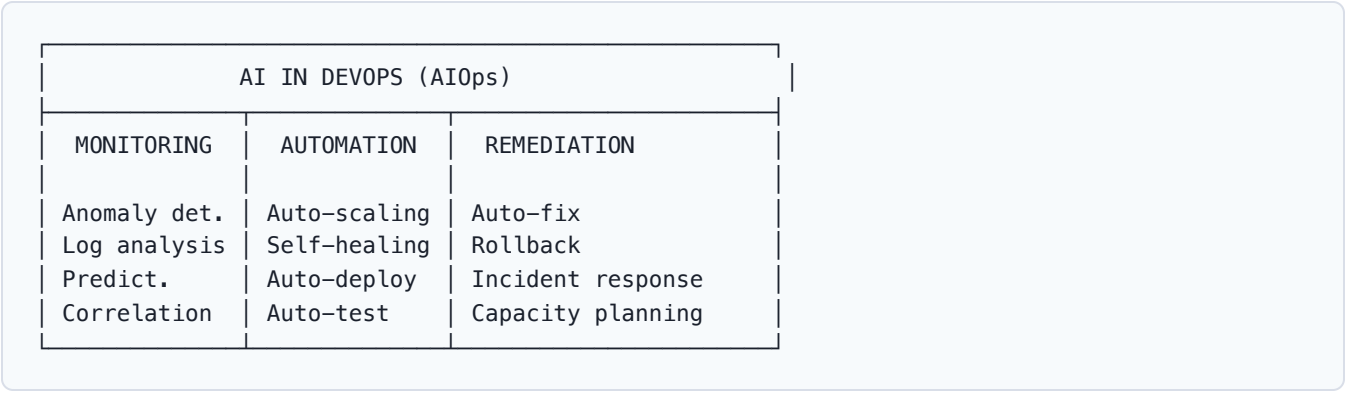
- MCP: Model Context Protocol
- Herramientas: Ecosistema de Herramientas
- DevOps: IA en DevOps
- Agentes: Agentes de IA

Ai Devops Integration

Integración de IA en DevOps

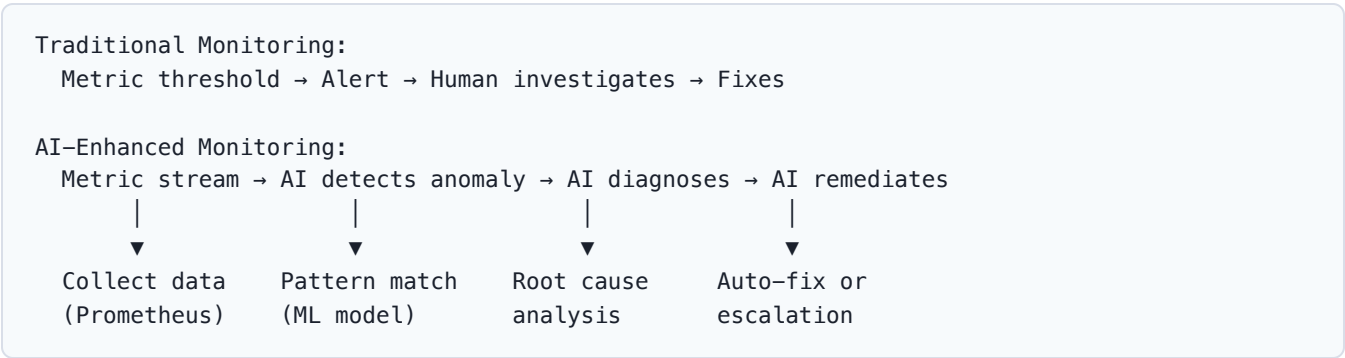
AIOps combina IA con prácticas DevOps para automatizar operaciones, detectar anomalías y remediar incidentes automáticamente.

AIOps Landscape



1. Intelligent Monitoring

AI-Enhanced Observability

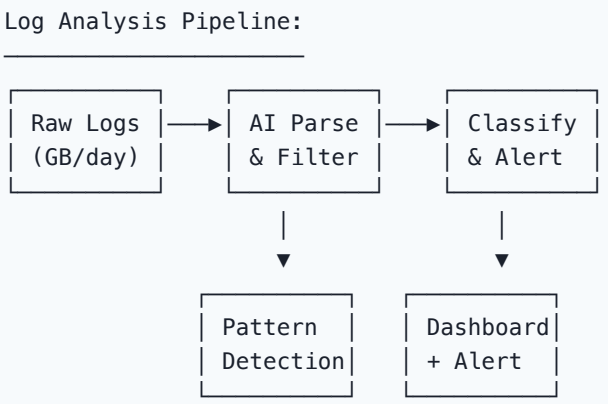


Anomaly Detection

```
# AI-powered anomaly detection for metrics
def detect_anomaly(metric_series: list[float]) -> dict:
    """Detect anomalies in metric time series."""
    import numpy as np

    mean = np.mean(metric_series[:-1])
    std = np.std(metric_series[:-1])
    current = metric_series[-1]
```

```
    z_score = (current - mean) / std if std > 0 else 0

    if abs(z_score) > 3:
        return {
            "is_anomaly": True,
            "severity": "critical" if abs(z_score) > 5 else "warning",
            "z_score": z_score,
            "expected": mean,
            "actual": current
        }
    return {"is_anomaly": False}
```

Log Analysis with AI



2. Self-Healing Systems

Auto-Remediation

```
Incident Response Flow:
─────────────────────────

1. DETECT
   ├── AI monitors metrics/logs
   ├── Anomaly detected
   └── Severity classified

2. DIAGNOSE
   ├── AI correlates events
   ├── Root cause identified
   └── Impact assessed

3. REMEDIATE
   ├── AI selects remediation action
   ├── Executes automated fix
   └── Validates fix worked
```

```
4. REPORT
   ├── Incident documented
   ├── Post-mortem generated
   └── Prevention suggested
```

Common Auto-Remediations

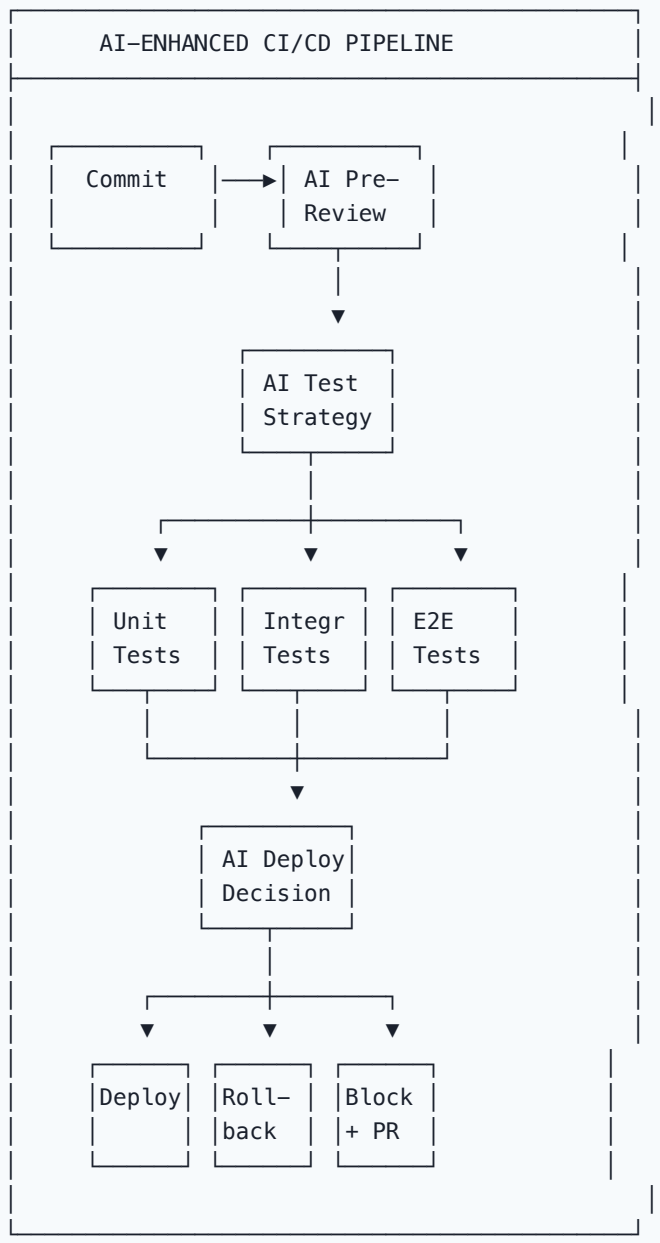
Issue	AI Detection	Auto-Remediation
High CPU	Usage > 90% for 5min	Scale up instances
Memory leak	Memory growing linearly	Restart process
Disk full	Usage > 85%	Clean logs/temp files
SSL expiring	Days to expiry < 14	Auto-renew cert
Failed health check	3 consecutive failures	Restart + notify
Traffic spike	2x normal in 5min	Auto-scale + CDN

Implementation

```
class AutoRemediation:
    def __init__(self):
        self.rules = {
            "high_cpu": self.scale_up,
            "memory_leak": self.restart_process,
            "disk_full": self.cleanup_disk,
            "ssl_expiring": self.renew_certificate,
        }

    async def handle_incident(self, incident: dict):
        # AI diagnosis
        diagnosis = await self.ai_diagnose(incident)

        if diagnosis.auto_remediable:
            action = self.rules[diagnosis.rule]
            result = await action(diagnosis.context)

            if result.success:
                await self.notify("Auto-remediated", diagnosis)
            else:
                await self.escalate(diagnosis)
        else:
            await self.escalate(diagnosis)
```

3. CI/CD with AI

AI-Enhanced Pipelines



AI Test Selection

```
def ai_select_tests(changed_files: list[str]) -> list[str]:
    """AI selects most relevant tests for changed code."""
    response = openai.chat.completions.create(
        model="gpt-4o",
        messages=[{
            "role": "system",
            "content": "Given changed files, select the most relevant test files. Return JSON ar
```

```
        }, {
            "role": "user",
            "content": f"Changed files: {changed_files}"
        }],
        response_format={"type": "json_object"}
    )

    return json.loads(response.choices[0].message.content)["tests"]
```

4. Capacity Planning

AI-Powered Forecasting

```
Capacity Forecasting:
─────────────────────
Historical data → AI model → Prediction → Action

Example:
Current: 80% CPU utilization
Forecast: Will reach 95% in 3 days
Action: Auto-scale from 3 to 5 instances
Cost impact: +$200/month
```

AIOps Tools

Tool	Category	AI Features
Datadog	Monitoring	AI anomaly detection, root cause
Dynatrace	AIOps	Davis AI engine, auto-remediation
New Relic	Observability	NRQL + AI insights
PagerDuty	Incident Mgmt	AI routing, automation
BigPanda	AIOps	Event correlation
Moogsoft	AIOps	Noise reduction, correlation
Grafana + ML	Monitoring	Forecasting, anomaly

Best Practices

1. **Start with observability**: Good data before AI
2. **Human in the loop**: AI suggests, human approves critical actions
3. **Gradual automation**: Start monitoring, then suggest, then auto-fix
4. **Test remediations**: Verify fixes don't cause new issues
5. **Cost awareness**: AI processing has costs

6. **Explainability**: AI decisions must be explainable

Relación con otros conceptos

- DevOps: DevOps e Infraestructura
- Scripts: Scripts y Automatización
- Agentes: Agentes de IA
- Métricas: Métricas de productividad
- Seguridad: IA en Seguridad

Futuro la Software

El Futuro de IA en Software Factories

Tendencias y predicciones para la IA en software factories durante 2025-2030.

Timeline de Evolución

```
2024-2025: AI-Assisted Era
├── Copilot ubiquity
├── Multi-model selection
├── Basic agentic coding
└── MCP adoption

2025-2026: Agentic Era
├── Autonomous coding agents
├── Multi-agent systems
├── AI-native IDEs
└── Full SDLC AI integration

2026-2027: Autonomous Era
├── AI handles entire features
├── Self-healing codebases
├── AI-driven architecture
└── Continuous AI review

2027-2028: Orchestration Era
├── AI manages projects
├── Cross-team AI coordination
├── AI-driven prioritization
└── Predictive development

2028-2030: Transformation Era
├── Software factory paradigm shift
├── New roles emerge
├── AI as primary developer
└── Human focus on strategy
```

Tendencias Clave

1. Modelos Especializados

```
Evolution of Models:
────────────────────

2023: General-purpose GPT-4
       ↓
```

```
2024: Code-specialized (Codestral, DeepSeek)
       ↓
2025: Task-specialized (reasoning, planning)
       ↓
2026: Domain-specialized (healthcare, finance)
       ↓
2028: Self-improving models
```

Trend	Impact	Timeline
Larger context windows	Entire codebase in context	Now
Multi-modal models	Code + diagrams + docs	Now
Reasoning models	Better planning	2025-2026
Small efficient models	On-device coding	2026-2027
Self-improving	Models that learn from feedback	2027-2028

2. Autonomous Agents

```
Agent Evolution:
─────────────────
L1 (Now):    Autocomplete, chat
L2 (2025):   Multi-file changes
L3 (2026):   Feature implementation
L4 (2027):   Project management
L5 (2028+):  Full autonomous SWE
```

3. AI-Native Development

```
Traditional:  Human writes code → AI assists
AI-Native:    Human defines intent → AI implements
              Human reviews → AI refines
              Human deploys → AI monitors
              Human learns → AI adapts
```

4. Multi-Agent Orchestration

```
2025: Single agent per task
       ↓
2026: Coordinated agent teams
       ↓
2027: Agent organizations
       ↓
2028: Self-organizing agent swarms
```

5. AI-Driven Quality

```
Quality Evolution:
──────────────────

2024: AI-assisted code review
       ↓
2025: AI-generated tests + review
       ↓
2026: AI-verified quality gates
       ↓
2027: Self-testing codebases
       ↓
2028: Zero-defect through AI
```

Nuevos Roles

Role	Description	Emerges
AI Engineer	Build AI-powered tools	2024
Prompt Engineer	Optimize AI interactions	2024
Agent Orchestrator	Design multi-agent systems	2025
AI Architect	AI system design	2025
AI Safety Engineer	Ensure AI reliability	2025
Human-AI Interaction Designer	Optimize human-AI workflows	2026
Autonomous Systems Manager	Manage agent fleets	2027
AI Ethics Officer	Ensure responsible AI use	2026

Impacto en la Software Factory

Organizational Changes

```
Traditional SF:              AI-Augmented SF:
──────────────               ──────────────────
Developer    10              Developer     5
QA Engineer   3              AI Engineer    2
DevOps        2              Agent Orch.    1
Architect     1              Architect      1
PM            1              PM             1
                             ──────────────────
Total:       17              Total:        10
Output:     baseline         Output:    3-5x
```

New Metrics

Metric	Traditional	AI-Enhanced
Velocity	Story points/sprint	Features/week
Quality	Defects/KLOC	Zero-defect rate
Lead time	Weeks	Hours
Deployment	Weekly	Continuous
Cost	Cost per feature	Cost per outcome

Implications

Para Developers

Aspect	Impact	Action
Coding skills	Less manual coding	Focus on architecture
Debugging	AI-assisted	Learn AI debugging
Testing	AI-generated	Focus on strategy
Documentation	Auto-generated	Focus on review
Learning	AI-guided	Continuous upskilling

Para Organizations

Aspect	Impact	Action
Hiring	Fewer juniors needed	Reskill existing team
Structure	Flatter hierarchy	New roles emerge
Process	More automated	Focus on strategy
Cost	Lower dev cost, higher tool cost	ROI measurement
Quality	Higher baseline	New quality standards

Risks y Mitigaciones

Risks

```
1. Over-reliance on AI
   └── Mitigation: Maintain human skills
```

```
2. Skill atrophy
   └── Mitigation: Regular training

3. Security concerns
   └── Mitigation: AI security scanning

4. Cost escalation
   └── Mitigation: Budget controls

5. Quality regression
   └── Mitigation: Human review gates

6. Ethical concerns
   └── Mitigation: Ethics framework
```

Ethical Considerations

Issue	Description	Mitigation
Job displacement	Fewer developers needed	Reskilling programs
Bias in AI	AI may have biases	Diverse training data
Transparency	Black box decisions	Explainable AI
Accountability	Who is responsible?	Clear policies
Privacy	Code sent to cloud	Self-hosted options

Preparing for the Future

For Individuals

1. **Learn AI tools**: Master current AI assistants
2. **Focus on architecture**: Understanding systems > writing code
3. **Build AI literacy**: Understand how AI works
4. **Specialize**: Domain expertise becomes more valuable
5. **Stay adaptable**: Technology changes fast

For Organizations

1. **Invest in AI**: Budget for tools and training
2. **Experiment now**: Start with pilots
3. **Measure impact**: Track ROI of AI
4. **Update processes**: Adapt workflows for AI
5. **Build AI culture**: Encourage AI adoption

Prediction Matrix

Prediction	Confidence	Timeline
80%+ devs use AI daily	High	2025
AI writes 50%+ of new code	High	2026
Autonomous feature implementation	Medium	2027
AI manages entire projects	Low-Medium	2028
Software factory paradigm shift	Medium	2030
Human developer role redefined	High	2028

Relación con otros conceptos

- Presente: IA en Ingeniería
- Agentes: Agentes de IA
- Limitaciones: Limitaciones y Riesgos
- Métricas: Impacto en Métricas
- Software Factory: ¿Qué es una Software Factory?

README

IA en Software

Cómo la inteligencia artificial está transformando la ingeniería de software: desde coding assistants hasta agentes autónomos y arquitecturas multi-agente.

Contenido

Fundamentos de IA (1-13)

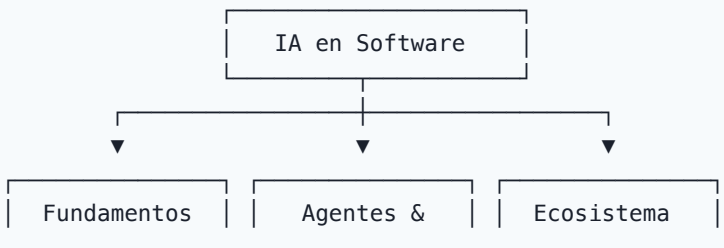
#	Archivo	Tema	Nivel	
01	[[../07-IA-Software/01-ia-en-ingenieria	IA en Ingeniería de Software]]	Panorama general, LLMs, impacto actual	Principiante
02	[[../07-IA-Software/02-copilot-efecto	GitHub Copilot y el Efecto Copilot]]	Funcionamiento, productividad, limitaciones	Intermedio
03	[[../07-IA-Software/03-generacion-codigo	Generación de Código con IA]]	Técnicas, calidad, review	Intermedio
04	[[../07-IA-Software/04-ai-test-generation	Generación de Tests con IA]]	Auto-generate tests, mutation testing	Intermedio
05	[[../07-IA-Software/05-ai-code-analysis	Análisis de Código con IA]]	Code review, vulnerabilidades, refactoring	Intermedio
06	[[../07-IA-Software/06-ai-documentation	IA en Documentación Técnica]]	Auto-doc, API docs, README	Principiante
07	[[../07-IA-Software/07-prompt-engineering-dev	Prompt Engineering para Devs]]	Técnicas, templates, best practices	Intermedio
08	[[../07-IA-Software/08-ai-pair-programming	AI Pair Programming]]	Workflows, tips, limitaciones	Intermedio
09	[[../07-IA-Software/09-ai-software-design	IA en Diseño y Arquitectura]]	Design suggestions, pattern detection	Avanzado
10	[[../07-IA-Software/10-ai-legacy-modernization	IA para Legacy Modernization]]	Code understanding, migration, refactoring	Avanzado
11	[[../07-IA-Software/11-ai-security-scanning	IA en Seguridad de Código]]	SAST with AI, vulnerability detection	Avanzado
12	[[../07-IA-Software/12-ai-limitaciones-riesgos	Limitaciones y Riesgos]]	Hallucinations, bias, security, dependency	Intermedio

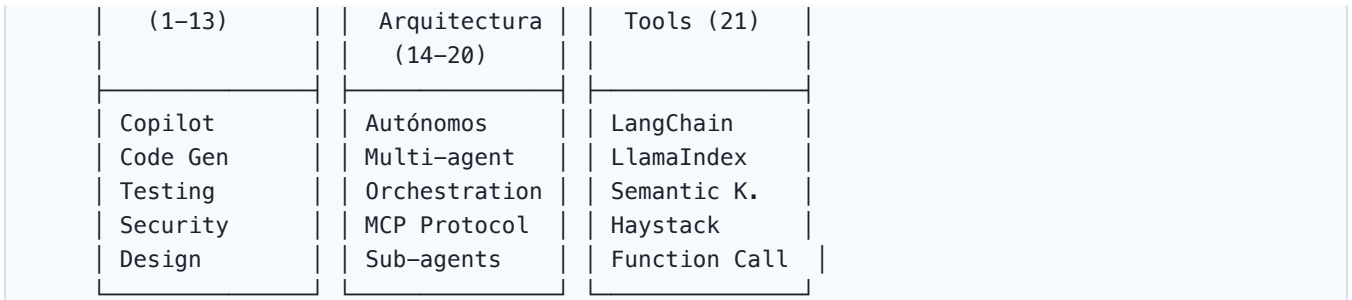
#	Archivo	Tema	Nivel	
13	[[../07-IA-Software/13-ai-metrics-impact	Impacto en Métricas]]	Before/after, ROI de AI tools	Intermedio

Agentes y Arquitecturas (14-24)

#	Archivo	Tema	Nivel	
14	[[../07-IA-Software/14-agentes-ia-vision-general	Agentes de IA: Visión General]]	Definición, componentes, lifecycle	Intermedio
15	[[../07-IA-Software/15-agentes-autonomos	Agentes Autónomos vs Asistidos]]	Autonomy levels L0-L5, use cases	Avanzado
16	[[../07-IA-Software/16-arquitectura-agentes	Arquitecturas de Agentes]]	ReAct, Plan-and-Execute, Reflexion, LATS	Avanzado
17	[[../07-IA-Software/17-sub-agentes-orquestacion	Sub-agentes y Orquestación]]	Delegation, coordination, supervisor	Avanzado
18	[[../07-IA-Software/18-multi-agent-systems	Sistemas Multi-agente]]	CrewAI, AutoGen, LangGraph, patterns	Avanzado
19	[[../07-IA-Software/19-mcp-protocol	Model Context Protocol (MCP)]]	Spec, transport, lifecycle, capabilities	Avanzado
20	[[../07-IA-Software/20-mcp-servers-tools	MCP Servers, Tools y Resources]]	Server architecture, tool definition	Avanzado
21	[[../07-IA-Software/21-ai-tools-ecosystem	Ecosistema de Herramientas AI]]	LangChain, LlamaIndex, Semantic Kernel	Intermedio
22	[[../07-IA-Software/22-ai-scripts-automation	Scripts y Automatización con IA]]	Custom tools, function calling	Intermedio
23	[[../07-IA-Software/23-ai-devops-integration	IA en DevOps]]	AIOps, intelligent monitoring	Avanzado
24	[[../07-IA-Software/24-futuro-ia-software	Futuro de IA en Software Factories]]	Trends 2025-2030, autonomous agents	Avanzado

Mapa conceptual





Secciones relacionadas

- 01 — Fundamentos
- 02 — Estructura
- 03 — Procesos
- 05 — Herramientas
- 06 — Métricas
- 08 — Calidad y Seguridad
- 09 — DevOps e Infraestructura